

第 1 章 C++类与对象

1.1 从 C 到 C++语言

在本教程前篇中，系统地介绍了 C 语言及其面向过程 (Procedure-Oriented Programming, POP) 的编程方法。C 是 1972 年由美国贝尔实验室的 D.M.Ritchie 研制成功的。目前许多著名的系统软件和应用软件都是用 C 语言编写的。但是随着软件规模的增大，用 C 语言编写程序的能力就渐渐变得力不从心了。在这样的背景下，由 AT&T Bell（贝尔）实验室的 Bjarne Stroustrup 等人在 20 世纪 80 年代初，以 C 语言为基础，成功地研发了 C++(C plus plus) 语言。C++保留了 C 语言原有的所有优点。同时提供了一个创造性特点，即 C++增加了面向对象程序 (Object-Oriented Programming, OOP) 设计的机制。

通过分析 C++的产生背景和过程，就可以很清晰地了解 C 和 C++语言之间关系：C 是 C++语言的子集，C++是 C 的超集。因此 C++完全与 C 兼容。也就是说，用 C 语言编写的程序几乎可以不加修改就可以在 C++编译器中进行编译。所以，C++既可以像 C 语言那样，采用面向过程的结构化程序设计思想来进行程序开发，又可用于面向对象的程序设计。可以说是一种功能强大的混合型的程序设计语言。C++对 C 的“增强”，最主要的表现在如下几点。

- (1) 在原来面向过程程序设计机制的基础上，对 C 语言的功能做了一些扩充。例如，变量的引用概念、例外处理机制等。
- (2) 增加了面向对象程序设计的机制。面向对象程序设计是针对大规模程序开发而提出的一种编程方法。其根本目标是提高软件开发的效率和代码重用。
- (3) 增加了泛型程序设计方法；例如，C++引入了函数模板、类模板和容器等编程方法。
- (4) 增加了多种程序数据保护安全机制；例如，C++引入了 `const` 属性变量，以及 I/O 数据类型检测机制，对象的封装性以及访问属性控制等机制。
- (5) 提高了代码重用性。例如，引入了运算符重载、函数重载、继承与派生等机制。

1.2 面向对象的程序设计方法

在本教材的前篇中，已经学习了面向过程的程序设计方法。下面将面向过程的程序设计方法的思想总结如下几个主要特点。

- 1) 设计范式：程序 = 算法 + 数据结构；
- 2) 设计思路：将程序看作是一个“事件”的过程，对过程进行“自顶而下，逐步细分”的模块（子过程）划分。这样就将一个程序按“功能”分解为多个子过程，在 C 语言中，子过程是通过函数来实现。因此，面向过程的程序设计基本思路是以算法（函数）为中心。
- 3) 设计特点：程序中的函数与数据是分离；数据都是公用的：即一个函数可以使用任何一组数据，而一组数据又能被多个函数所使用。整个程序以 `main()` 为中心，通过函数之

间的函数调用，从而来“串成”程序。面向过程的程序组成结构如图 1.1 所示。

4) 设计核心问题：算法（函数）确定和数据定义。

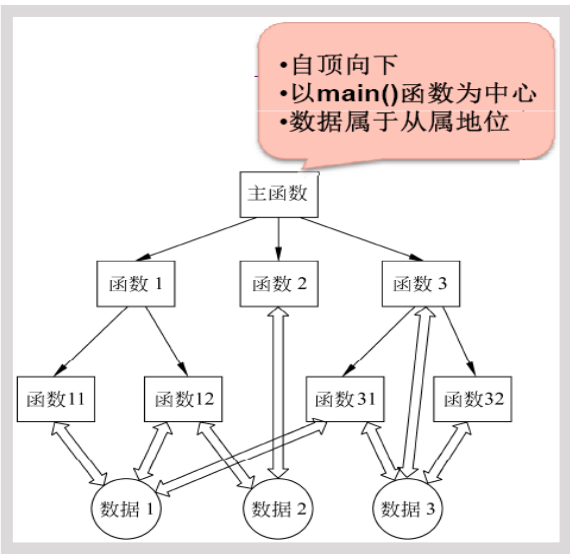


图 1.1 面向过程的程序组成结构

长期编程实践证明面向过程程序设计方法存在一些问题：

- (1) 难以满足大型复杂程序设计；对程序进行修改、扩展复杂；程序代码的重用性较差等；
- (2) 程序中的数据安全性保障机制弱；
- (3) 程序调试和测试过程比较困难和繁杂；
- (4) 程序设计的组织思想与现实世界的实际结构有些不吻合。

为了对面向过程程序编程与面向对象程序编程之间的差别有一个直观的认识，下面通过一个程序实例来对比说明。

实例 1-1 采用面向过程程序编程方法编写一个有关“栈操作”的程序。

```
#include <stdio.h>
#define STACK_SIZE 100
struct Stack
{
    int top;
    int buffer[STACK_SIZE];
};

bool stack_push(struct Stack *s, int i)
{
    if (s->top == STACK_SIZE-1)
    {
        printf("Stack is overflow.\n");
        return false;
    }
}
```

```

        else
        {
            s->buffer[++(s->top)] = i;
            return true;
        }
    }

bool stack_pop(struct Stack *s, int *i)
{
    if (s->top == -1)
    {
        printf("Stack is empty.\n");
        return false;
    }
    else
    {
        *i = s->buffer[(s->top)--];
        return true;
    }
}

int main()
{
    struct Stack st1, st2;
    int x;
    st1.top = -1;
    st2.top = -1;

    stack_push(&st1, 12); stack_pop(&st1, &x);
    stack_push(&st2, 20); stack_pop(&st2, &x);

    return 0;
}

```

实例 1-2 采用面向对象程序编程方法，重新编写程序来实现与程序实例 1-1 完全相同的功能。

```

#include <iostream>
using namespace std;
#define STACK_SIZE 100
class Stack
{
    int top;
    int buffer[STACK_SIZE];
public:

```

```

Stack() { top = -1; }
bool push(int i);
bool pop(int& i);
};
bool Stack::push(int i)
{
    if (top == STACK_SIZE-1)
    {
        cout << "Stack is overflow.\n";
        return false;
    }
    else
    {
        buffer[++top] = i;
        return true;
    }
}
bool Stack::pop(int& i)
{
    if (top == -1)
    {
        cout << "Stack is empty.\n";
        return false;
    }
    else
    {
        i = buffer[top--];
        return true;
    }
}
int main()
{
    Stack st1,st2;
    int x;
    st1.push(12); st1.pop(x); st2.push(20); st2.pop(x);
    return 0;
}

```

从上述两段程序实例可以看出，C++面向对象程序编程方法有如下特征。

(1) 引入了一种称为“类(class)”的数据结构。该自定义的数据类型将一组相关的数据以及对其操作的一组函数封装在一起。例如，实例 1-2 中声明了一种“Stack”类，在该类中，封装了“int top”和“int buffer[STACK_SIZE]”2 种数据类型，同时还封装了与这些数据相关的 2 个函数“bool push(int i)”和“bool pop(int& i)”。因此，

可以将这种称为“类(class)”的用户自定义数据类型理解为是对“结构体(struct)”数据类型的发展。在前篇中,我们学的“结构体”数据类型只是将一组相关数据封装在一起,而没有将与这些数据相关的函数封装进来。

2) 引入了“对象”(object)概念。对象可以理解为具有某一“类”数据类型特征的变量。因此,对象类似程序 1-1 中采用 Stack 结构体类型定义的结构体变量 st1 和 st2。实例 1-2 中的“st1”和“st2”就是采用“Stack”类构建的 2 个对象。

3) 引入了“对象的消息”的概念。所谓“对象的消息”就是通过“对象”来调用的函数。这些“消息”体现了对象之间的相互作用机制。例如,在实例 1-2 中,在 main 函数中采用“st1.push(12);”和“st2.push(20);”语句分别调用对象“st1”和对象“st2”的消息 push()。因此,我们也称“st1.push(12);”和“st2.push(20);”语句为主函数 main() 向对象“st1”和“st2”发“push()”消息。

通过对比分析程序实例 1-1 和 1-2,我们也可以归纳出 C++面向对象程序设计过程大致分为 2 个阶段。

第 1 步,类的封装(声明);设计程序所需的各种数据类,即决定把哪些数据和操作(算法)封装在一起;

第 2 步,消息调用。采用类来构建程序所需的对象及其消息调用。编写 main() 来实现对象间消息交互。消息交互(函数调用)来实现对象之间通信。各个对象的操作完成,整体软件的任务也就完成了。

通过上述对比分析,可以将面向对象的程序设计的思想总结为如下几个方面。

- (1) 设计范式: 程序 = 对象+消息; 如图 1.2 所示。
- (2) 基本思路: 将程序看作是有各种事物(对象)组成的系统;任何系统都是由若干对象组成,对象之间通过消息作用构成一个有序系统(软件)。整个程序设计时以数据(对象)为中心。
- (3) 基本特点: 对象有两大特征,分别是静态属性(数据)和动态行为(方法/函数);而且对象属性(数据)和行为(函数)是捆绑的,即属性和行为是对象不可分离两特征。
- (4) 核心问题: 类(对象)的设计与封装。

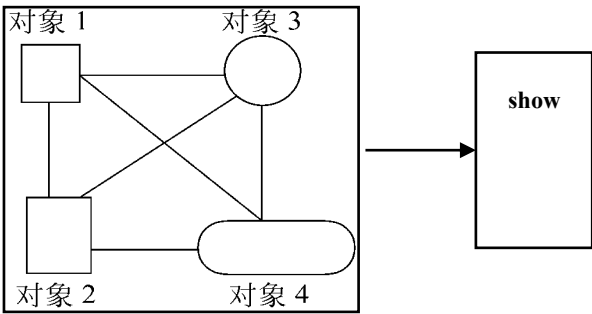


图 1.2 面向对象的程序组成

1.3 类与对象

在现实世界中，任何一个系统都是由许多对象组成的。例如，一辆汽车可以理解为由发动机、汽车底盘、电子控制装置、车轮等对象组成的系统；一次教学活动是由学生、教师、教学设施等对象组成的系统，等等。同样，一套程序也可理解为多个对象组成的系统。因此，面向对象的程序设计方法，真实地反映了自然界的万事万物的生成法则，完全符合自然界组成规律。那么，如何定义程序中的对象呢？这是面向对象程序设计的首要任务。

1.3.1 类和对象的概念

对象是构成一个系统的基本单位。任何具备属性(attribute)和行为(behavior)两种要素的事物都可看成对象(object)，例如，学生甲和教师乙等。对象可大可小，可以根据系统设计的要求，将一个系统划分为不同大小的对象，进而，通过对象之间的发送和接收消息，来实现对象之间互相联系和交互，从而来构成一个完整系统。

同理，在 C++ 语言中，一套程序也可以理解为由不同的对象及其相互作用而构成的系统。C++ 的对象都是由数据（属性）和函数（方法）两部分组成。

例如，在实例 1-2 中主要定义了 2 个对象（st1 和 st2）构成。

那么，如何在 C++ 语言中描述一个对象呢？为此，C++ 语言引入了“类”的概念。

类(class)是 C++ 新增加的重要的数据类型，也是 C++ 对 C 的最重要的发展。有了“类”的概念，就可以实现面向对象程序设计方法中的封装、继承、多态等三大重要的特性。

因此，“类”可以理解为是一种用户自定义的数据类型，它用来描述相同对象的共同特征。也就是说：“类”是“对象”的抽象(abstraction)；“对象”是“类”的具体实例(instance)。例如，可以声明一个“大学生”类来描述所有大学生的共同特征，“大学生甲”或“大学生乙”是具有“大学生”类特征的具体学生个体。

关于“类”与“对象”两者关系，类似于结构体类型和结构体变量。前面学习过，一般是先声明一个“结构体类型”，再用“结构体类型”去定义若干“结构体变量”。因此，一个结构体类型也可以理解为同类的结构体变量的描述。另外，在声明结构体类型时，不分配内存空间，只有定义了结构体变量才分配内存空间。同样，“类”是“对象”的抽象和描述。声明类不分配内存；对象是类具体的实现，在构建一个对象时，需要为其分配相应大小的内存空间。

1.3.2 类的声明方法

类(class)可以广义地理解为一种用户自定义的组合数据类型，也就是某种对象的类型。如果程序中要用到“类”类型，必须自己根据需要进行“类的声明”。当然，用户也可以使用第三方声明好的类，包括 C++ 编译系统等提供的一些标准类库中的类。

有前面的类比分析可以知道：类与结构体类型很相似，但类对结构体进行了 2 个方面的发展。第一、类不仅是多种数据的组合，还包含操作数据的函数。第二、类对这些数据和函数的访问属性进行了限定。因此，类声明一般格式如下：

```
class 类名
{ public :
    公用的数据;
    公用的成员函数;
  private :
    私有的数据;
    私有的成员函数;
};
```

其中,private 和 public 称为成员访问限定符(member access specifier)。除了 private 和 public 之外，还有一种成员访问限定符 protected (受保护的)，用 protected 声明的成员称为受保护的成员。private 数据成员和成员函数只能被本类的成员函数所调用。public 数据成员和成员函数不仅能被本类的成员函数所调用，还能在类外通过对象来调用。除此，还有 protected 访问限定符，有关 protected 成员性质在后面章节中介绍。

在声明“类”类型时，理论上，声明为 private 的成员和声明为 public 的成员的次序任意。关键字 private 和 public 可以分别出现多次，每个部分的有效范围到出现另一个访问限定符或类体结束时（最后一个右花括号）为止。另外，如果在类体中没有指明 private 或 public，则默认为 private。但是为了使程序清晰，良好编程素养建议如下：

(1) 使每一种成员访问限定符在类定义体中只出现一次。

(2) 先写 public 部分，把 private 部分放在类体的后部。这样可以使用户将注意力集中在能被外界调用的成员上，使阅读者的思路更清晰一些。

(3) 用户把自己或本单位经常用到的类放在一个专门的类库中，采用专门的头文件来声明类，需要使用这些类定义对象时，只需要将该头文件包含在相应的*.cpp 中，然后使用类直接定义对象等。这样就减少了程序设计的工作量，程序结构也很清晰。

实例 1-3 在某头文件 employee.h 中声明一个类 Employee。并在程序 employee.cpp 文件中采用 Employee 类定义了 2 个对象。

```
// employee.h 头文件
class Employee      //声明类的类型 Employee
{
  public :           //声明以下部分为公用的
    void show()
    {
      cout<<"num:"<<num<<endl; //在类内访问私有数据成员 num
      cout<<"name:"<<name<<endl;
      cout<<"sex:"<<sex<<endl;
    }
  private :         //声明以下部分为私有的
```

```

        int num;
        char name[20];
        char sex;
    };
//employee.cpp 程序文件
#include <iostream>
#include "employee.h"
using namespace std;

int main()
{
    Employee emp1, emp2;    //定义了两个 Employee 类的对象
    emp1.show();            //在类外通过对象 stud1 来访问 show()
    emp2.show();
    return 0;
}

```

在实例 1-3 中，show() 是公有成员函数，可以在类内和类外访问；而 num、name、sex 是私有数据成员，只能在类内访问，例如，被 show() 访问。

1.3.3 对象的定义

有了类声明后，就可以按照编程需求来定义一些对象。只有在构建（定义）一个对象之后，编译系统才会为这些对象分配存储空间，以存放对象中成员的值。对象定义方法类似变量的定义。可以归纳下列 3 种方法。

1. 方法一、先声明类类型，再定义对象。在 C++ 中，声明了“类”后，构建对象有两种形式。

(1) class 类名 对象名

例如：class Employee emp1, emp2; //把 class 和 Employee 合起来作为类名来定义对象。

(2) 类名 对象名

例如：Employee emp1, emp2; //直接用类名定义对象

这两种方法是等效的。第 1 种方法是从 C 语言继承下来的，第 2 种方法是 C++ 的特色，显然第 2 种方法更为简捷方便。

2. 方法二、类声明和对象定义同时完成。

```

class Employee    //声明类类型
{public:          //先声明公用部分
void show( )
{cout<<" num:" <<num<<endl;
  cout<<" name:" <<name<<endl;
  cout<<" sex:" <<sex<<endl;}
}

```



```
private :      //后声明私有部分
int num;
char name[20];
char sex;
}emp1, emp2;  //定义了两个 Employee 类的对象
```

上述实例在定义 Employee 类的同时，定义了 2 个 Employee 类的对象。

3. 方法三、直接定义对象。声明类时忽略类名。

```
class      //无类名
{private : //声明以下部分为私有的
:
:
public : //声明以下部分为公用的
:
:
}emp1, emp2; //定义了两个无类名的类对象
```

直接定义对象，在 C++ 中是合法的、允许的，但却很少用。在实际的程序开发中，一般都采用上面 3 种方法中的第 1 种方法。

1.3.4 对象的引用

采用一个类定义了一些对象后，在程序中就可以引用（使用）这些对象。注意，引用对象也就是引用对象中的成员，包括：成员函数和数据成员。在类外引用对象中的成员可以有 3 种方法。

1. 通过对象名和成员运算符来引用对象中的成员。

一般形式为：对象名. 成员名

其中，成员名包括数据成员和成员函数。只要是类中公共成员，都可以通过对象来类外引用。但 private 成员不能通过对象在类外引用。

例如，在程序中可以写出以下语句：

```
emp1.num=1001; //假设 num 已定义为公用的整型数据成员
emp1.sex= 'F' ; //假设 sex 是私有数据成员，不能被外界引用
emp1.show( ); //正确，调用对象 stud1 的公用成员函数
show( ); //错误，没有指明是哪一个对象的 show 函数
```

其中“.”是成员运算符，用来对成员进行限定，指明所访问的是哪一个对象中的成员。注意不能只写成员名而忽略对象名。在上例中，由于没有指明对象名，编译时把 show（）作为普通函数处理。

2. 通过指向对象的指针来引用对象中的成员。

采用某个类定义一个对象后，该对象就分配相应内存空间。因此，可以定义一个某种类型的指针变量来指向该对象的内存空间首地址。然后，就可以用指针来引用对象中的成员。例如：

```
class Date
```

```
{
public : //数据成员是公用的
int month;
int day;
};
Date d,*p; //定义对象 d 和指针变量 p
p=&d;//使 p 指向对象 d
cout<<p->month;//输出 p 指向的对象中的成员 hour
在 p 指向 d 的前提下, p->month, (*p).month 和 d.month 三者等价。
```

3. 通过对象的引用变量来引用对象中的成员。

如果为一个对象定义了一个引用变量。引用和原变量名共同标记同一段存储空间,实际上它们是同一个对象的 2 个名称而已。因此,完全可以通过引用变量来引用对象中的成员。

例如,假设已声明了 Date 类,构建了 d1 对象,则:

```
Date d1; //定义对象 d1
Date &d2=d1;//定义 Date 类引用变量 d2, 并使之初始化为 d1
cout<<d2.month;//输出对象 d1 中的成员 month
由于 d2 与 d1 共占同一段存储单元(即 d2 是 d1 的别名),因此 d2.month 等同于 d1.month。
备注: 有关对象的指针变量和对象的引用变量的概念将在本章第 6 节中将详细讨论。
```

1.4 类的成员函数

在类中声明的函数称为该类的成员函数(简称类函数)。而在任何类之外声明的函数是非成员函数(简称为普通函数)。类的成员函数的定义、使用及作用与前面介绍的普通函数基本上是一样的,它也有返回值、函数类型和函数参数等属性。但成员函数与普通函数也存在一些区别,主要体现为如下几点。

(1) 成员函数是属于一个类的成员,只能在类体中声明,但函数的定义可以在类体内或类外进行。例如,实例 1-3 中的 Employee 类中公有成员函数 show() 是在类内中定义的。

(2) 成员函数有访问属性限定。可以被指定为 private(私有的)、public(公用的)或 protected(受保护的)。这些访问属性限定了成员的调用它的权限(它能否被调用)。例如私有的成员函数只能被本类中的其它成员函数所调用,而不能被类外调用。但成员函数(无论私有还是公有)都可以访问本类中任何成员(包括私有的和公用的)。

一般的做法是将需要被外界调用的成员函数指定为 public,它们是类的对外接口。不需要被外界调用的成员函数指定为 private,这些 private 函数只能被本类中的其他成员函数所调用。因此, private 成员函数是类中其它成员的工具函数(utility function)。

1.4.1 成员函数的声明与定义

在声明类时，对类中成员函数声明格式和普通函数声明格式相同，如下所示。

函数类型 函数名（参数列表及类型）；

至于成员函数的定义，既可以在类体中定义，也可以在类体中只写成员函数的声明，而在类体外面对函数进行定义。例如，对于实例 1-3 中定义的成员函数，如果改写为函数外定义，则其代码如实例 1-4 所示。

实例 1-4，Employee 类中的成员函数 show() 在类外进行定义。

```
//头文件：employee.h

class Employee      //声明类类型 Employee
{
public :             //声明以下部分为公用的
    void show();    //公用成员函数原型声明
private :           //声明以下部分为私有的
    int num;
    char name[20];
    char sex;
};

//在类外定义 show 类函数
void Employee::show()
{
    cout<<"num:"<<num<<endl; //函数体
    cout<<"name:"<<name<<endl;
    cout<<"sex:"<<sex<<endl;
}

// employee.cpp
#include <iostream>
using namespace std;
#include "employee.h"

int main()
{
    Employee emp1, emp2;    //定义了两个 Employee 类的对象
```

```

    emp1.show();           //在类外通过对象 stud1 来访问 show()
    emp2.show();
    return 0;
}

```

通过对比分析实例 1-3 和 1-4 中的程序代码，可以总结如下 2 点规则。

(1) 在类体中直接定义函数时，不需要在函数名前面加上类名（为什么？）。但成员函数在类外定义时，必须在函数名前面加上类名和作用域限定符(field qualifier) “::”。

“::” 也称为作用域运算符，用它声明函数是属于哪个类的。

(2) 如果在作用域运算符 “::” 的前面没有类名，或者函数名前面既无类名又无作用域运算符 “::”，如 ::show() 或 show()，则表示 show 函数不属于任何类，这个函数不是成员函数，而是普通函数。

另外，良好编程素养是：在类体内部对成员函数作声明，而在类体外部定义成员函数。对于一般函数体比较长的成员函数，更是建议在类外定义。

1.4.2 inline 成员函数

在教材前面章节中，已经介绍过 C++ 程序的函数调用机制和过程。因为调用时会在主调函数中产生断点保护、程序执行权从主调函数到被调函数的转移、以及主调函数断点恢复等过程，因此，函数调用时需要一定的时间和空间的开销图。1.3 表示函数调用的过程。

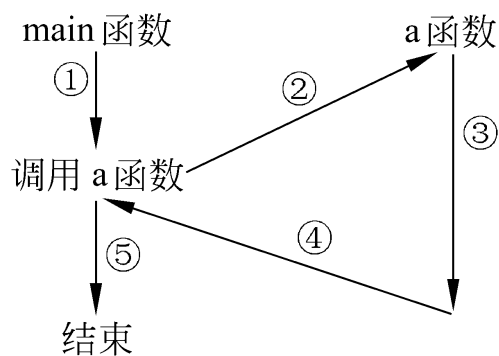


图 1.3 函数调用过程

C++ 提供了一种提高程序执行效率的方法，即在编译时将所调用函数的代码直接替换到主调函数的代码中，而不是将执行流程转出去。这种替换到主调函数中的函数称为内置函数 (inline function)，也称为内嵌函数。

那么如何声明内置函数呢？其基本方法是在函数首行的左端加一个关键字 inline 即可。

实例 1-5. 内置函数 min 声明。

```

#include <iostream>
using namespace std;

```

```

inline int min(int, int, int); //声明函数,注意左端有 inline
int main()
{
    int i=10, j=20, k=30, m;
    m=min(i, j, k);
    cout<<"min="<<m<<endl;
    return 0;
}

inline int min(int a, int b, int c) //定义 min() 为内置函数
{
    if(b<a) a=b; //求 a, b, c 中的最大者
    if(c<a) a=c;
    return a;
}

```

由于在定义函数时声明为内置函数，因此编译系统在遇到函数调用“min(i, j, k)”时，就用 min 函数体的代码代替“min(i, j, k)”，同时将实参代替形参。这样，程序第 6 行“m=min(i, j, k);”就被置换成

```

if (j<i) i=j;
if (k<i) i=k;
m=i;

```

注意：可以在声明函数和定义函数时同时写 inline，也可以只在声明时写 inline，定义时可省略，这样也能按内置函数处理。注意：内置函数中不能包括复杂的控制语句，如循环语句和 switch 语句等。

那么，类的成员函数是否也可以声明为内置函数呢？答案是肯定的。

在 C++ 程序中，为了减少时间开销，如果在类体中定义的成员函数中不包括循环等控制结构，C++ 系统会自动将它们作为内置 (inline) 函数来处理。也就是说，在程序调用这些成员函数时，并不是真正地执行函数的调用过程 (如保留返回地址等处理)，而是把函数代码嵌入程序的调用点。因此，C++ 要求对一般的内置函数要用关键字 inline 声明，但对类内定义的成员函数，可以省略 inline，因为这些成员函数已被隐含地指定为内置函数。例如，对于实例 1-3 中的 show() 成员函数，已被隐含地指定为内置函数。

实例 1-6. show() 成员函数为隐含内置函数

```

//头文件

class Employee      //声明类的类型 Employee
{
public :             //声明以下部分为公用的
    void show()      //show 函数为隐含内置函数
    {

```

```

        cout<<"num:"<<num<<endl; //在类内访问私有数据成员 num
        cout<<"name:"<<name<<endl;
        cout<<"sex:"<<sex<<endl;
    }
private :           //声明以下部分为私有的
    int num;
    char name[20];
    char sex;
};

```

```

// employee.cpp
#include <iostream>
#include "employee.h"
using namespace std;

int main()
{
    Employee emp1,emp2;    //定义了两个 Employee 类的对象
    emp1.show();           //在类外通过对象 stud1 来访问 show()
    emp2.show();
    return 0;
}

```

其中第 3 行 void show() 也可以写成 inline void show()。

将 show 函数显式地声明为内置函数。以上两种写法是等效的。对在类体内定义的函数，一般都省略写 inline。但在类体外定义，系统并不把它默认为内置(inline)函数，调用这些成员函数的过程和调用一般函数的过程是相同的。如果想将这些成员函数指定为内置函数，应当用 inline 作显式声明。例如，对于实例 1-4 中的成员函数 show()，如果是在类外定义的，且需要将其定义内置函数，则程序需要修改为实例 1-7 所示。

实例 1-7. 类外定义的 show() 需要显式声明为内置函数。

```

//employee.h
class Employee      //声明类类型 Employee
{
public :             //声明以下部分为公用的
    inline void show(); //声明此成员函数为内置函数
private :           //声明以下部分为私有的

```

```

        int num;
        char name[20];
        char sex;
    };

    inline void Employee::show( ) // 在类外定义 show 函数为内置函数
    {
        cout<<"num:"<<num<<endl; //函数体
        cout<<"name:"<<name<<endl;
        cout<<"sex:"<<sex<<endl;
    }

//employee.cpp
#include <iostream>
using namespace std;
#include "employee.h"

int main()
{
    Employee emp1,emp2;    //定义了两个 Employee 类的对象
    emp1.show();           //在类外通过对象 emp1 来访问 show()
    emp2.show();
    return 0;
}

```

注意：如果在类体外定义 inline 函数，则必须将类的定义和成员函数的定义都放在同一个头文件中（或者写在同一个源文件中）；否则编译时无法进行置换。但是这样做，又不利于类的接口与类的实现分离，也不利于类的信息隐蔽。引入内置函数，虽然程序的执行效率提高了，但从软件工程质量的角度来看，却破坏了良好的程序结构。两者平衡是根据程序设计优化目标而特别考虑。

1.4.3 成员函数的存储方式和 this 指针

使用类来构建（定义）对象时，系统会为每一个对象分配一定大小的内存空间。假设用同一个类定义了 10 个对象，那么是否需要分别为 10 个对象的数据和函数代码分配独立的内存空间吗？

答案是否定。C++编译系统会为这 10 个对象的数据成员分别分配独立的内存空间，但只用一段空间来存放这个 10 对象的共同函数代码。在调用各对象的函数时，都去调用这个公用的函数代码。如图 1.4 所示。

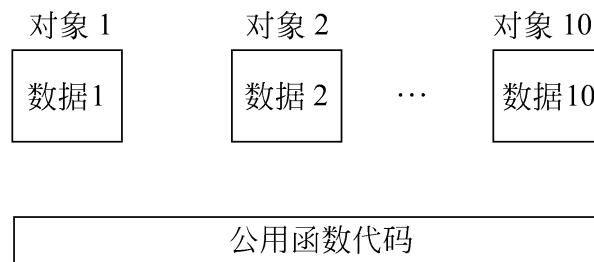


图 1.4 对象的数据和函数代码的分配方法

显然，这样做会大大节约存储空间。因此，每个对象所占用的存储空间只是该对象的数据部分所占用的存储空间，而不包括函数代码所占用的存储空间。采用下面有方法可以进行验证。声明了一个类（例如 `class employee`），该类中包含一些数据成员和成员函数。然后采用“`cout<<sizeof(employee)<<endl;`”语句来测试该类对象所占用的字节数。测试结果表明：一个对象所占的空间大小只取决于该对象中数据成员所占的空间，而与成员函数无关。

根据对象内存分配机制，有同学会问：不同对象的成员函数代码相同，但是为什么执行结果是不相同的？这就需要了解 C++ 的 `this` 指针机制。C++ 为一个类中每个成员函数专门设立了一个 `this` 的指针。`this` 指向本类对象，它的值是当前被调用的成员函数所在的对象的起始地址。假如，对于实例 1-4 程序中定义的 `employee` 类，假设采用 `employee` 类定义了 2 个对象，分别为 `emp1` 和 `emp2`。如果有语句 `emp1.show()`，该语句应该是显示对象 `emp1` 中的 `num`、`name` 和 `sex`；而 `emp2.show()`，则是引用对象 `emp2` 中的 `num`、`name` 和 `sex`。这是因为编译系统在编译 `emp1.show()` 语句时，对象 `stud1` 的起始地址赋给 `this` 指针。在成员函数引用数据成员时，按照 `this` 找到对象 `emp1` 的数据成员。因此，实际上的 `show()` 函数体隐式表达形式为：

```
Employee::show()
{cout<<" num:" << this->num<<endl;
 cout<<" name:" << this->name<<endl;
 cout<<" sex:" << this->sex<<endl;}
```

由于 `this` 指向当前对象，因此 `emp1.show()` 函数体相当于执行：

```
cout<<" num:" << (emp1.num)<<endl;
cout<<" name:" << (emp1.name)<<endl;
cout<<" sex:" << (emp1.sex)<<endl;
}
```

而 `emp2.show()` 函数体相当于执行：

```
cout<<" num:" << (emp2.num)<<endl;
```



```
cout<<" name:" << (emp2.name)<<endl;
cout<<" sex:" << (emp2.sex)<<endl;
}
```

1.5 类的构造函数和析构函数

在用类构建对象时，需要对每个对象的内存空间进行初始化。例如，需要对对象中的数据成员赋初值，其作用类似于变量的初始化。但是，在 C++ 语言中，考虑到对象成员的访问属性等限定，专门引入一种新的机制（类的构造函数）来完成对象的初始化工作。相对应，在对象使用完成后，需要释放内存等资源，C++ 也专门引入了析构函数机制来完成对象释放清理工作。

1.5.1 对象的初始化与构造函数

C++ 提供了构造函数 (constructor) 来处理对象的初始化。构造函数是类中一种特殊的成员函数，它与其他成员函数不同的特殊性表现为如下 4 点。

(1) 不需要用户来调用它，而是在构建对象时自动执行。

(2) 构造函数的名字必须与类名同名，不能由用户任意命名，以便编译系统能识别它并把它作为构造函数处理。

(3) 它不具有任何类型，没有返回任何值。

(4) 构造函数的功能是由用户定义的，用户根据初始化的要求设计函数体和函数参数。如果用户没有为某个类定义构造函数，编译系统会为其产生一个缺省构造函数。且缺省构造函数的函数体为空，即不执行任何操作。

实例 1-8 类的构造函数

```
//date.cpp
#include <iostream>
using namespace std;
class Date
{
public :
    Date()
    {
        year=2018;
        month=1;
        day=1;
    }
}
```

```

        void show_date( );
private :
    int year;
    int month;
    int day;
};

void Date::show_date( )
{
    cout<<year<<"/"<<month<<"/"<<day<<endl;
}

int main()
{
    date dt1;
    dt1.show_date();
    return 0;
}

```

程序运行结果为：2018/1/1

1.5.2 构造函数类型

根据对象构建的不同需求，C++编译系统定义了多种类型的构造函数。具体包括：

- (1) 系统默认构造函数（即空构造函数）：如果用户自己没有定义构造函数，则 C++ 系统会自动生成一个构造函数，只是这个构造函数的函数体是空的，也没有参数，不执行初始化操作。
- (2) 用户自定义构造函数。这类函数进一步可以分为“不带参数的构造函数”和“带参数的构造函数”两类。对于带参数的构造函数，根据参数传递方式不同，还可以进一步分为如下 3 种情况。
 - 1) 函数体传递参数；
 - 2) 通过参数初始化表来传递；
 - 3) 指定默认参数。

下面就重点介绍带参数构造函数的定义方法。

- (1) 带参数的构造函数。

在实例 1-8 中定义了一个不带参数的构造函数。构造函数在函数体中对数据成员赋初值。使用无参构造函数会使得该类构建的每一个对象都会初始化一组数值，这就无法体现对

象初始值的个性化需求。因为有时用户希望对不同的对象赋予不同的初值。

要解决上述问题，可以采用带参数的构造函数。有参构造函数在构建不同对象时，从外面将不同的数据传递给构造函数，以实现多个对象的不同初始化。

有参构造函数首部的一般格式为：构造函数名(类型 1 形参 1，类型 2 形参 2，…);

那么，在使用有参构造函数构建对象时，如何将外面将不同的数据传递给构造函数呢？因为用户是不能调用构造函数的，无法采用常规的调用函数方式给出实参。于是，C++规定构造函数的实参是在定义对象时给出的。下面给出了有参构造函数构建对象的一般格式为：

类名 对象名(实参 1，实参 2，…);

实例 1-9 编一个在类中用带参数的构造函数构建对象的程序。

```
#include <iostream>

using namespace std;

class Cube
{
public :
    Cube(int, int, int); //声明带参数的构造函数
    int volume();
private :
    int height;
    int width;
    int length;
};

Cube::Cube(int h, int w, int len) //在类外定义带参数的构造函数
{
    height=h;
    width=w;
    length=len;
}

int Cube::volume() //定义计算体积的函数
{
    return (height*width*length);
}

int main()
```

```

{
    Cube cube1(10, 20, 30);    //建立对象 cube1, 并指定 cube1 长、宽、高的值
    cout<<"The volume of cube1 is "<<cube1.volume()<<endl;
    Cube cube2(15, 25, 35);    //建立对象 cube2, 并指定 cube2 长、宽、高的值
    cout<<"The volume of cube2 is "<<cube2.volume()<<endl;
    return 0;
}

```

通过上述实例可以归纳如下特点:

- (1) 带参数的构造函数中的形参, 其对应的实参在定义对象时给定。
- (2) 这种方法是通过函数体的赋值语句来实现对不同的对象进行不同的初始化。

另外, C++还提供一种“参数初始化表”的有参数构造函数, 用来实现对对象数据成员的初始化。其特点是不在函数体内通过赋值语句对数据成员初始化, 而是在函数首部通过“参数初始化列表”来实现, 从而来简化构造函数的函数体。

例如, 实例 1-9 中定义构造函数可以改用以下形式:

```
Cube::Cube(int h, int w, int len):height(h), width(w), length(len){ }
```

其中“height(h), width(w), length(len)”称为参数初始化列表。

因此, 带“参数初始化表”的有参数构造函数的一般声明格式为:

【类名::】函数名(参数列表): 初始化列表{ }

1.5.3 构造函数的重载

所谓函数重载(function overloading)是指对一个函数名重新赋予它新的含义, 用同一函数名定义多个函数。这些函数的参数个数和参数类型不同。例如, int max(int a, int b, int c); 和 double max(double a, double b, double c); 这 2 个函数功能不同, 分别为求 3 个整数中的最大者和求 3 个双精度数中最大者, 因此, 参数类型和返回值也不同。但函数名相同。这就是 C++语言引入的重载机制。

C++这种函数重载机制也适用于构造函数。在一个类中可以定义多个构造函数, 以便对类对象提供不同的初始化的方法, 供用户选用。这些构造函数具有相同的名字, 而参数的个数或参数的类型不相同。

实例 1-10 类中声明了多个构造函数

```

//date.cpp
#include <iostream>
#include <iomanip>
using namespace std;
class Date
{
private:

```

```

        int day, month, year;
public:
    Date(int dd, int mm, int yy) : day(dd), month(mm), year(yy) { };
    Date(int dd, int mm) : day(dd), month(mm) { year = 2018; };
    Date(int dd) : day(dd) { month = 1; year = 2018; };
    Date() { day = 1; month = 1; year = 2018; };
    Date(const char * strDate);
    ~Date();
    void show() { cout<<setw(4)<<setfill('0')<<year<<'-'
                    <<setw(2)<<setfill('0')<<month<<'-'
                    <<setw(2)<<setfill('0')<<day<<endl; };
};

```

```

Date::Date(const char * strDate)
{
    int i = 0;
    int buffer[3] = {0};

    for(int cnt = 0; strDate[cnt] != '\0'; cnt++)
    {
        if(strDate[cnt] == '-')
        {
            i++;
        }
        else
        {
            buffer[i] = buffer[i] * 10 + strDate[cnt] - '0';
        }
    }

    year = buffer[0];
    month = buffer[1];
    day = buffer[2];
}

```

```

Date globalDay;

```

```

int main()
{
    Date today(1);
    Date Oct1(1, 10);
    Date RDay(1, 9, 2015);
    Date now;
    Date guy("1990-2-20");
}

```

```
        return 0;
    }
```

在实例 1-10 的程序中定义了 5 个重载的构造函数,其实还可以定义其他重载构造函数。在主函数中构建 5 个不同对象,分别调用相应的构造函数来实现初始化。

建议:(1)分析这些对象分别调用哪些构造函数?(2)试着完成这些构造函数定义代码。

说明:

(1) 调用时不必给出实参的构造函数,称为默认构造函数(default constructor)。显然,无参的构造函数属于默认构造函数。注意,一个类只能有一个默认构造函数。

(2) 如果在建立对象时选用的是无参构造函数,应注意正确书写定义对象的语句。

(3) 尽管在一个类中可以包含多个构造函数,但是对于每一个对象来说,建立对象时只执行其中一个构造函数,并非每个构造函数都被执行。

1.5.4 使用默认参数的构造函数

一般情况下,在函数调用时形参从实参那里取得值。因此,实参的个数应与形参相同。C++还提供一种使用默认参数机制,即在函数声明时,给形参一个默认值,这样形参就不必一定要从实参取值了。例如 `int max1(int a=1,int b=2, int c=3);` 其中 3 个参数都给予默认值。如果在调用此函数时,确认使用全部或部分参数的默认值,则可以不给出这些实参的值。如果不想使形参取此默认值,则通过实参另行给出。

```
max1(6, 2, 5); // 实际是 max1(6, 2, 5);
```

```
max1(6, 4); //实际是 max1(6, 4, 3);
```

```
max1(6); //实际是 max1(6, 2, 3);
```

```
max1(); //实际是 max1(1, 2, 3);
```

可以看到:在调用有默认参数的构造函数时,实参的个数可以与形参的个数不同。如果实参未给定的,从形参的默认值得到值。利用这一特性,可以使函数的使用更加灵活。另外,用带有默认参数的函数可以减少或者不用重载函数。

因为实参与形参的结合是从左至右顺序进行的。因此指定默认值的参数必须放在形参表列中的最右端,否则出错。例如:

```
void f1(float a, int b=0,int c, char d=' a' ); //不正确
```

```
void f2(float a, int c, int b=0, char d=' a' ); //正确
```

在 C++语言,默认参数的机制也同样适用于类的构造函数。在构造函数中形参也可以指定为某些默认值,即如果用户不指定实参值,编译系统就使形参取默认值。

实例 1-11,采用默认值参数的构造函数,将实例 1-10 中的 Date 类的 4 个重载构造函

数合并为一个构造函数，并实现同样的功能。

```
//date.cpp
#include <iostream>
#include <iomanip>
using namespace std;

class Date
{
private:
    int day, month, year;
public:
    Date(int dd = 1, int mm = 1, int yy = 2018) : day(dd), month(mm), year(yy)
{ }; //初始化 d, m, y
    Date(const char * strDate); //用字符串日期初始化
    ~Date();

    void show() { cout<<setw(4)<<setfill('0')<<year<<'-'
                    <<setw(2)<<setfill('0')<<month<<'-'
                    <<setw(2)<<setfill('0')<<day<<endl; };
};

Date::Date(const char * strDate)
{
    int i = 0;
    int buffer[3] = {0};

    for(int cnt = 0; strDate[cnt] != '\0'; cnt++)
    {
        if(strDate[cnt] == '-')
        {
            i++;
        }
        else
        {
            buffer[i] = buffer[i] * 10 + strDate[cnt] - '0';
        }
    }
}
```

```

        }
    }

    year = buffer[0];
    month = buffer[1];
    day = buffer[2];
}

int main()
{
    Date today(1);
    Date Oct1(1, 10);
    Date RDay(1, 9, 2015);
    Date now;
    Date guy("1990-2-20");

    return 0;
}

```

可以看到：在构造函数中使用默认参数是方便而有效的，它提供了建立对象时的多种选择，它的作用相当于好几个重载的构造函数。也就是说，默认参数构造函数是利用 C++ 对默认参数的支持，可以将多个版本的构造函数合并为一个。另外，它还有一个好处是：即使在调用构造函数时没有提供实参值，不仅不会出错，而且还确保按照默认的参数值对对象进行初始化。尤其在希望对每一个对象都有同样的初始化状况时用这种方法更为方便。

说明：

- (1) 应该在声明构造函数时指定默认值，而不能只在定义构造函数时指定默认值。
- (2) 如果构造函数的全部参数都指定了默认值，则在定义对象时可以给一个或几个实参，也可以不给出实参。
- (3) 在一个类中定义了全部是默认参数的构造函数后，不能再定义重载构造函数。

1.5.5 类的析构函数

析构函数 (destructor) 也是一个特殊的成员函数，它的作用与构造函数相反。它的名字是类名的前面加一个位取反运算符 “~” 符号。从取名规则可以体会到：析构函数是与构造函数作用相反的函数。

当对象的生命期结束时，会自动执行析构函数。具体来说，在下列情况下程序就会执行析构函数：

(1) 如果在一个函数中定义了一个对象(它是自动局部对象), 当这个函数被调用结束时, 对象应该释放, 在对象释放前自动执行析构函数。

(2) static 局部对象在函数调用结束时对象并不释放。因此, 也不调用析构函数, 只在 main 函数结束或调用 exit 函数结束程序时, 才调用 static 局部对象的析构函数。

(3) 如果定义了一个全局对象, 则在程序的流程离开其作用域时(如 main 函数结束或调用 exit 函数)时, 调用该全局对象的析构函数。

(4) 如果用 new 运算符动态地建立了一个对象, 当用 delete 运算符释放该对象时, 先调用该对象的析构函数。

析构函数的作用并不是删除对象, 而是在撤销对象占用的内存之前完成一些清理工作, 使这部分内存可以被程序分配给新对象使用。析构函数也是一种特殊的类成员函数, 其特点为:

(1) 析构函数不返回任何值, 没有函数类型, 也没有函数参数。因此, 它不能被重载。一个类可以有多个构造函数, 但只能有一个析构函数。

(2) 实际上, 析构函数的作用并不仅限于释放资源方面。它还可以被用来执行“用户希望在最后一次使用对象之后所执行的任何操作”。例如, 输出有关的信息。

(3) 如果用户没有定义析构函数, C++编译系统会自动生成一个析构函数, 但它只是徒有析构函数的名称和形式, 实际上什么操作都不进行。

实例 1-12 包含缺省构造函数和析构函数的 C++程序

```
//date.cpp
#include <iostream>
#include <iomanip>
using namespace std;

class Date
{
private:
    int day, month, year;
public:
    void show() { cout<<setw(4)<<setfill('0')<<year<<'-'
                    <<setw(2)<<setfill('0')<<month<<'-'
                    <<setw(2)<<setfill('0')<<day<<endl; };
//Date(); 编译系统自动生成缺省构造函数
//~Date();编译系统自动生成缺省析构函数
};
```

```

int main()
{
    Date A;//隐藏地先调用 A.Date() 初始化对象 A;
    return 0;
}    //隐藏地先调用 A.~Date() 释放资源

```

实例 1-13 包含构造函数和析构函数的 C++ 程序。

// Employee.cpp

```
#include<string>
```

```
#include<iostream>
```

```
using namespace std;
```

```
class Employee //声明 Employee 类
```

```
{public :
```

```
    Employee(int n, string nam, char s ) //定义构造函数
```

```
{
```

```
    num=n;
```

```
    name=nam;
```

```
    sex=s;
```

```
    cout<<"Constructor called."<<<endl; //输出有关信息
```

```
}
```

```
~Employee() //定义析构函数
```

```
{cout<<"Destructor called."<<<endl;} //输出有关信息
```

```
void show() //定义成员函数
```

```
{
```

```
    cout<<"num: "<<num<<endl;
```

```
    cout<<"name: "<<name<<endl;
```

```
    cout<<"sex: "<<sex<<endl<<endl;
```

```
}
```

```
private :
```

```
    int num;
```

```
    string name;
```

```
    char sex;
```

```
};

int main()
{
    Employee emp1(10010, "Wang_li", 'f'); //建立对象 emp1
    emp1.show(); //输出员工的数据
    Employee emp2(10011, "Zhang_fun", 'm'); //定义对象 emp2
    emp2.show(); //输出员工的数据
    return 0;
}
```

程序运行结果如下：

Constructor called.

num: 10010

name:Wang_li

sex: f

Constructor called.

num: 10011

name:Zhang_fun

sex:m

Destructor called.

Destructor called.

(执行 emp1 的构造函数) (执行 emp1 的 show 函数)

(执行 emp2 的构造函数) (执行 emp2 的 show 函数)

(执行 emp2 的析构函数) (执行 emp1 的析构函数)

通过分析实例 1-13 的执行结果可以发现：调用析构函数的次序正好与调用构造函数的次序相反，即最先被调用的构造函数，其对应的（同一对象中的）析构函数最后被调用，而最后被调用的构造函数，其对应的析构函数最先被调用。如图 1.5 示意。

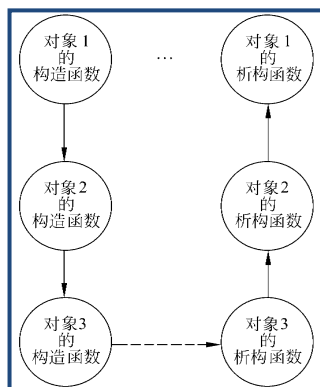


图 1.5 构造函数和析构函数的执行顺序

但是，并不是在任何情况下都是按这一原则处理的。因为对象和前面介绍的变量一样，存在着作用域和存储类别等属性。对象可以在不同的作用域中定义，可以有不同的存储类别。例如，在实例 1-14 中，分别构建了全局对象 `globalday`、局部对象 `today`，以及动态临时无名对象等。这些会影响调用构造函数和析构函数的时机。

实例 1-14 对象的作用域和存储类型

```
//DATE.CPP

#include <iostream>
#include <iomanip>
using namespace std;

class Date
{
private:
    int day, month, year;
public:
    Date(int dd = 1, int mm = 1, int yy = 2018) : day(dd), month(mm), year(yy)
    { }; //初始化 d, m, y

    Date(const char * strDate);
    //用字符串日期初始化

    ~Date();

    void show() { cout<<setw(4)<<setfill('0')<<year<<'-'
                  <<setw(2)<<setfill('0')<<month<<'-'
                  <<setw(2)<<setfill('0')<<day<<endl; };
};
```

```

Date::Date(const char * strDate)
{
    int i = 0;
    int buffer[3] = {0};

    for(int cnt = 0; strDate[cnt] != '\0'; cnt++)
    {
        if(strDate[cnt] == '-')
        {
            i++;
        }
        else
        {
            buffer[i] = buffer[i] * 10 + strDate[cnt] - '0';
        }
    }

    year = buffer[0];
    month = buffer[1];
    day = buffer[2];
}

```

```

Date globalDay;//构建全局对象

```

```

int main()
{
    Date today; //构建局部自动对象
    Static Date today1; //构建局部静态对象
    Date *pDate = new Date; //构建局部动态临时对象
    delete pDate;
    return 0;
}

```

下面归纳一下什么时候调用构造函数和析构函数。

(1) 在全局范围中定义的对象（即在所有函数之外定义的对象，如 globalday），它的构造函数在文件中的所有函数（包括 main 函数）执行之前调用。但如果一个程序中有多个文件，而不同的文件中都定义了全局对象，则这些对象的构造函数的执行顺序是不确定的。当 main 函数执行完毕或调用 exit 函数时（此时程序终止），调用析构函数。

(2) 如果定义的是局部自动对象（例如在函数中定义对象，如 today），则在建立对象时调用其构造函数。如果函数被多次调用，则在每次建立对象时都要调用构造函数。在函数调用结束、对象释放时先调用析构函数。

(3) 如果在函数中定义静态(static)局部对象，则只在程序第一次调用此函数建立对象时调用构造函数一次，在调用结束时对象并不释放，因此也不调用析构函数，只在 main 函数结束或调用 exit 函数结束程序时，才调用析构函数。

(4) 如果在函数中使用 new 运算符，构建了一个动态临时对象，其生命期从 new 运算符开始，此时调用构造函数。在执行的指针采用 delete 运算符释放时，对象生命期结束，此时调用析构函数。

1.6 对象数组和对象指针

1.6.1 对象数组

前面我们学习过简单变量构成的数组。其实，对象也可以构建一个数组结构，对象数组就是若干同类的对象集合。

在日常生活中，有许多实体的属性是共同的，只是这些实体的属性值不同而已。例如，某公司有 40 名员工，每个员工的属性包括姓名、性别、年龄、工号等。如果为每 1 个员工建立 1 个对象，这时可以定义 1 个“员工类”对象数组，每一个数组元素是一个“员工类”对象。

例如 `Employee em[40];` //假设已有 Employee 类，定义一个长度为 40 的 em 对象数组

上例中，在构建数组 em 时，同样要调用构造函数。因为有 40 个元素，则需要调用 40 次构造函数。那么如何对这些对象进行初始化呢？一般方法是在花括号中分别写出带有实参的构造函数。如果没有提供参数，则调用时无参构造函数。

例如

```
Employee em[3]={ //定义对象数组
Employee(1001,25,1),    //调用第 1 个元素的构造函数，为它提供 3 个实参
Employee(1002,27,2),    //调用第 2 个元素的构造函数，为它提供 3 个实参
Employee(1003,22,3)     //调用第 3 个元素的构造函数，为它提供 3 个实参
};
```

因此，可以归纳出一般格式如下。

类名 对象名[长度]={构造函数（实参列表 1），构造函数（实参列表 2），...}

在建立对象数组时，分别调用构造函数，对每个元素初始化。而且，每一个元素的实参分别用括号包起来，对应构造函数的一组形参。

实例 1-15 对象数组的构建与初始化。

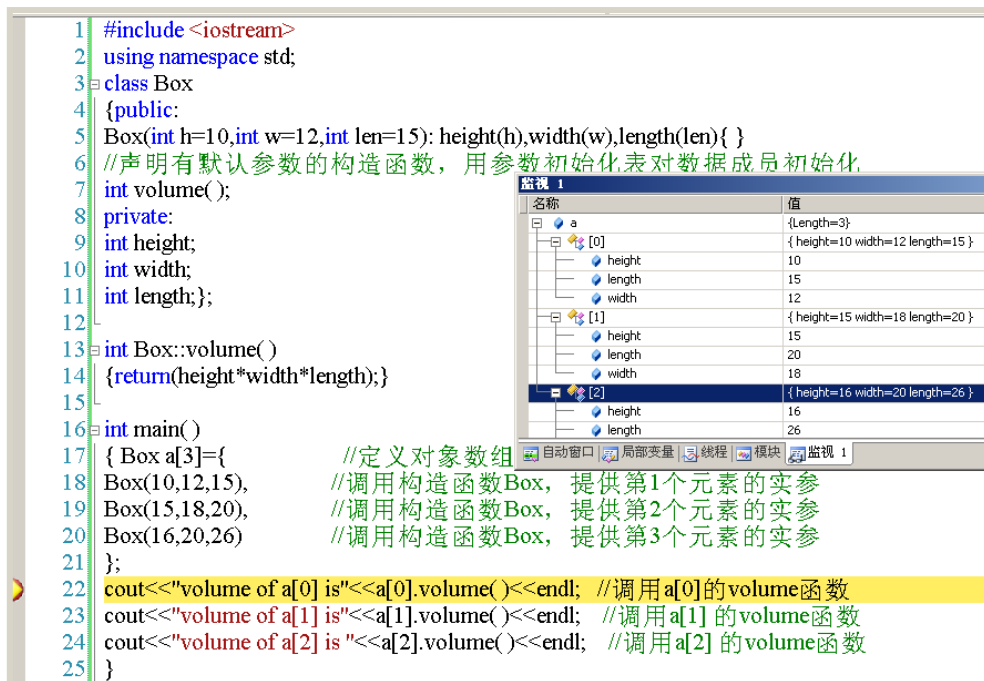


图 1.6 对象数组的构建与初始化

在上述代码实例中，在第 22 条语句设立的一个断点，断点调试的结果如上断点调试视图。从图中可以看出，程序运行到第 22 语句时，已经在内存中构建了 3 个对象，并且每个对象都有了确定的初始化。

1.6.2 对象指针

(1) 对象的相关指针类型

前面介绍过，当程序构建一个对象时，需要给该对象在内存中分配一定大小连续空间，用来保存对象中的成员函数和数据成员。这些数据和函数所占用的内存单元的起始地址被称为相关的指针（如图 1.7 所示）。因此，一个对象的相关指针类型有：对象指针，如：对象 box1 的指针为 0x0012ff58，表示为&box1；对象 box1 的数据成员 width 的指针为 0x0012ff5c，表示为&(box1.width)；对象 box1 的成员函数 volume() 的指针为 0x004115b0 等。

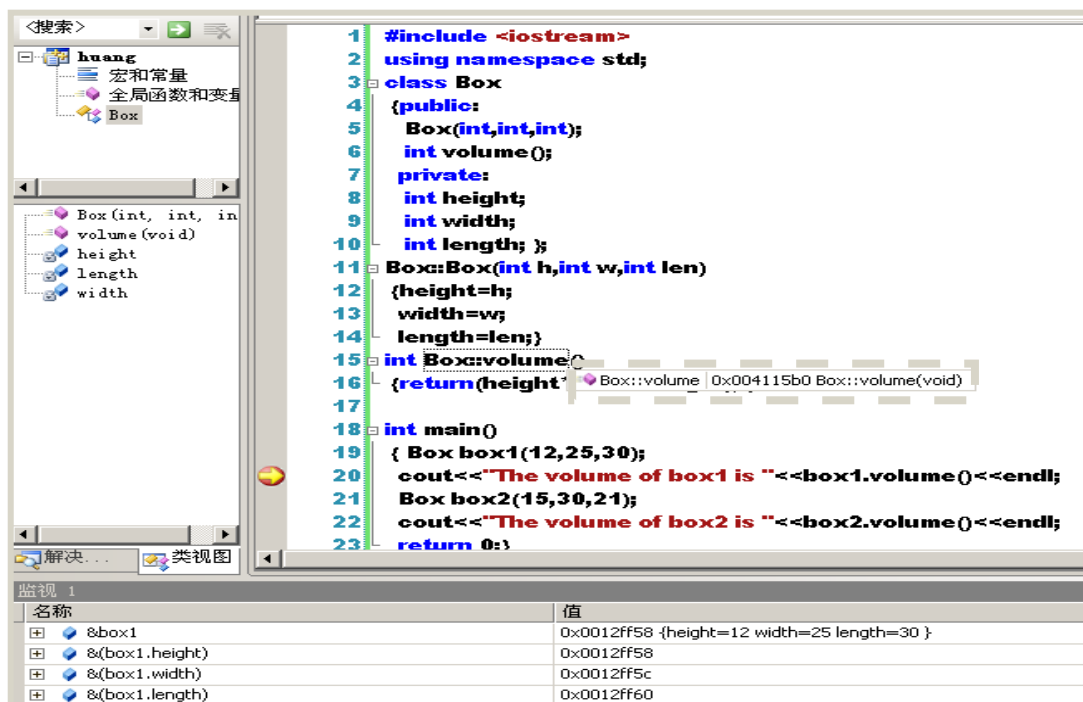


图 1.7 对象的指针

因此，我们可以定义相应类型的指针变量来保存对象的不同类型指针。下面就分别介绍对象的指针变量。

(2) 指向对象的指针变量

在建立对象时，编译系统会为每一个对象分配一定的存储空间。对象空间的起始地址就是对象的指针。可以定义一个指针变量，用来存放对象的指针。

例如 `employee em(1001, 25, 1);`

`employee *pt=&em;` //pt 是指向对象 em 的指针变量

实例 1-16 对象的指针变量及基于指针变量对对象的引用方式

```
#include <iostream>
using namespace std;
class Date
{
public:
    Date(int d, int m, int y)
    {
        day = d;
        month = m;
        year = y;
    }
    void get_date();
private:
    int day;
    int month;
    int year;
```



```

};

void Time::get_time()
{
    cout<<hour<<' :'<<minute<<' :'<<sec<<endl;
}

int main()
{
    Time *pt;           //定义 pt 为指向 Time 类对象的指针变量
    Time t1(12, 39, 23); //定义 t1 为 Time 类对象
    pt = &t1;           //将 t1 的起始地址赋给 pt
    pt->get_time();
    (*pt).get_time();
    return 0;
}

```

在该实例中，分别了构建对象 t1，以及定义了指向对象 t1 的指针变量 pt。并通过指针变量 pt 引入对象 t1 的成员函数 get_time()。

因此，我们在此可以归纳出指向类对象的指针变量的一般形式如下。

类名 *对象指针名；

还可以归纳出通过对象指针访问对象和对象的成员的一般形式如下。

(*指针变量名). 数据成员 (或成员函数)；或者：指针变量名->数据成员 (或成员函数)；

(3) 指向对象成员的指针

对象有地址，对象中的成员也有地址，存放对象成员地址的指针变量就是指向对象成员的指针变量。

定义指向对象数据成员的指针变量的方法和定义指向普通变量的指针变量方法相同。如实例 1-17 所示。

需要提醒读者注意：定义指向对象成员函数的指针变量的方法和定义指向普通函数的指针变量方法有所不同。因为成员函数是某类中的一个成员。因此，编译系统要求，成员函数的指针变量的类型在以下 3 方面都要匹配：

- 1) 函数参数的类型和参数个数；
- 2) 函数返回值的类型；
- 3) 所属的类。

因此，可以归纳出定义指向公用(public)成员函数的指针变量的一般形式如下。

数据类型名 (类名::*指针变量名) (参数表列)；

该针变量的初始化方法如下。

指针变量名=&类名::成员函数名；

实例 1-17 对象成员的指针变量及引用方式

```

#include <iostream>

```

```

using namespace std;
class clock
{
public:
    clock(int h, int m, int s)
    {
        hour = h;
        minute = m;
        sec = s;
    }
    void get_clock();
//private:
    int hour;
    int minute;
    int sec;
};

void clock::get_clock()
{
    cout<<hour<<' :'<<minute<<' :'<<sec<<endl;
}

int main()
{
    clock *pt;
    clock t1(12, 39, 23);
    int *p1 = &t1.hour;           //定义指针变量p1 指向对象 t1 的数据成员 hour
    *p1 = 15;                     //通过 p1 来修改 hour
    pt = &t1;                     //将 t1 的起始地址赋给 pt
    pt->get_clock();
    void (clock::*p3) ();         //定义指向 clock 类成员函数的指针变量 p3
    p3 = &clock::get_clock;      //使 p3 指向 Time 类公用成员函数 get_clock
    (t1.*p3) ();
    return 0;
}

```

如果上述程序 1-17 中的 clock 类的数据成员是私有的。即 private 没有注释掉，情况会如何？

1.7 共用数据的保护

C++相对于 C 来说，最大的特点是提高了数据的安全型，引入多种有效的数据安全保护措施。例如：设置了对象成员的 private 访问属性等，还引入了对象的常变量属性。

所谓常变量属性(const)是指某一变量在初始化之后，只能被读取，不能被修改。这样

使得变量既能在一定范围内共享，又保证它不被任意修改。例如，`const int a=12;` 表示常变量 `a` 在初始化为 12 之后，在程序中任何地方都只能读取，不能被修改。

1.7.1 常对象及常成员函数

在常变量概念的基础上，C++引入了常对象的概念。凡希望保证某对象中的数据成员不被改变，则可以声明该对象为常对象。常对象定义的一般形式如下。

类名 `const` 对象名[(实参表列)]; 或者 `const` 类名 对象名[(实参表列)];

在定义对象时指定对象为常对象。常对象必须要有初值，如 `clock const t1(1, 10, 6);` 这样，在所有的场合中，对象 `t1` 中的所有成员的值都不能被修改。

需要特别注意：如果一个对象被声明为常对象，则不能调用该对象的非 `const` 型的成员函数(除了由系统自动调用的隐式的构造函数和析构函数)。

例如，对于实例 1-17 中已定义的 `Time` 类，如果有 `const clock t1(1, 5, 6);`

若在程序中出现：`t1.get_clock();` 该语句是错误的。因为企图调用常对象 `t1` 中的非 `const` 型成员函数 (`get_clock();`) 是非法的。这是为了防止这些函数可能修改常对象中数据成员的值。

考虑到程序设计者无意中会修改常对象中的数据成员的情况，因此，编译系统引入了对象的常成员函数概念。常成员函数专门用来访问常对象中的数据成员。编译系统只检查函数的声明，只要发现调用了常对象的成员函数，而且该函数未被声明为 `const`，就报错，以此来提示编程者注意。

对象的常成员函数的定义一般格式如下。

函数类型 函数名(参数类型及列表) `const` ;

注意 `const` 的位置在函数名和括号之后，`const` 是函数类型的一部分，在声明函数和定义函数时都要有 `const` 关键字，在调用时不必加 `const`。

例如：`void get_clock() const ;`

如果将成员函数声明为常成员函数，则只能引用本类中的数据成员，而不能修改它们，例如只用于输出数据等。常成员函数可以引用 `const` 数据成员，也可以引用非 `const` 的数据成员。

特别说明：

(1) 不要误认为常对象中的成员函数都是常成员函数。如果在常对象中的成员函数未加 `const` 声明，编译系统把它作为非 `const` 成员函数处理。

(2) 常对象只保证其所有的数据成员是常数据成员(见后面说明)，其值不被修改。

(3) 还有一点要指出：常成员函数不能调用另一个非 `const` 成员函数。

在编程时有时一定要修改常对象中的某个数据成员的值。ANSI C++考虑到实际编程时的需要，引入 `mutable` 机制，就是将需要修改的数据成员声明为 `mutable`，如 `mutable int count;` 把 `count` 声明为可变的数据成员，这样就可以用声明为 `const` 的成员函数来修改它

的值。

1.7.2 对象的常数据成员

如果只需要将对象中部分数据进行常变量属性保护,这样就没有必要将整个对象定义为常对象。只需要将对象中部分成员声明为 `const`,即对象的常数据成员。常数据成员其作用和用法与一般常变量相似,用关键字 `const` 来声明常数据成员。例如:在 `Time` 类中,如果要将数据成员 `hour` 声明为常数据成员,其格式为:`const int hour`。

常数据成员的值是不能改变的。因此,只能通过构造函数的参数初始化表对常数据成员进行初始化。不能采用在构造函数中对常数据成员赋初值的方法。

即使在类外定义构造函数,应通过参数初始化表对常数据成员 `hour` 初始化,例如,如果需要 `clock` 类中的常数据成员 `hour` 初始化,构造函数在类外定义时,其格式如下
`clock::clock(int h):hour(h) {}`

另外需要说明,`const` 数据成员可以被 `const` 成员函数引用,也可以被非 `const` 的成员函数引用。

同理,常对象的数据成员都是常数据成员,因此,常对象的构造函数只能用参数初始化表对常数据成员进行初始化。

既然 C++ 同时引入了常对象和对象的常数据成员,那么在程序设计时,如何选择常对象和常数据成员呢?基本原则可以归纳如下:

(1) 如果在一个类中,有些数据成员的值允许改变,另一些数据成员的值不允许改变,则可以将一部分数据成员声明为 `const`,以保证其值不被改变。此时,可以用非 `const` 的成员函数读取这些数据成员的值,或者修改非 `const` 数据成员的值。当然也可以采用常成员函数来读取常数据成员。

(2) 如果要求所有的数据成员的值都不允许改变,则将对象声明为 `const` (常对象)。此时,只能用 `const` 成员函数引用数据成员,这样起到“双保险”的作用,切实保证数据安全。因为:如果已定义了一个常对象,只能调用其中的 `const` 成员函数,而不能调用非 `const` 成员函数(不论这些函数是否会修改对象中的数据)。

1.7.3 指向对象的常指针和指向常对象的指针变量

1. 指向对象的常指针

我们知道:指针变量也是变量。如果希望指针变量在初始化之后也不被改变,则可以将指针变量也声明为 `const` 型。定义指向对象的常指针的一般形式如下。

类名 * `const` 指针变量名;

注意:在定义指针变量时使之初始化。

例如: `clock t1(1,2,5), t2; //定义 2 个对象`

`clock * const pt1=&t1;` //定义一个指向 `clock` 类型的常指针变量 `pt1`, `pt1` 指向对象 `t1`, 此后不能再改变指向;

`pt1=&t2;` //错误, `pt1` 不能改变指向。

需要特别说明:

(1) 指向对象的常指针变量的值不能改变, 即始终指向同一个对象, 但其所指向对象(如 `t1`) 的值可以改变。

(2) 如果想将一个指针变量固定地与一个对象相联系(即该指针变量始终指向一个对象), 可以将它指定为 `const` 型指针变量。

2. 指向常对象的指针变量。

定义指向常变量(常对象)的指针变量的一般形式如下。

`const` 类型名 *指针变量名;

例如: `const char *ptr;` //定义了一个指向常变量的指针变量 `ptr`

注意 `const` 的位置在最左侧, 它与类型名 `char` 紧连, 表示指针变量 `ptr` 指向的 `char` 变量是常变量, 不能通过 `ptr` 来改变其值的。

需要说明:

(1) 如果一个变量已被声明为常变量, 只能用指向常变量的指针变量指向它, 而不能用一般的(指向非 `const` 型变量的)指针变量去指向它。此时不能通过此指针变量改变该变量的值。

(2) 指向常变量的指针变量除了可以指向常变量外, 还可以指向未被声明为 `const` 的变量。

(3) 如果函数的形参是指向非 `const` 型变量的指针, 实参只能用指向非 `const` 变量的指针, 而不能用指向 `const` 变量的指针, 这样, 在执行函数的过程中可以改变形参指针变量所指向的变量(也就是实参指针所指向的变量)的值。

如果函数的形参是指向 `const` 型变量的指针, 在执行函数过程中显然不能改变指针变量所指向的变量的值, 因此, 允许实参是指向 `const` 变量的指针, 或指向非 `const` 变量的指针。

以上的对应关系与在(2)中所介绍的指针变量和其所指向的变量的关系是一致的: 指向常变量的指针变量可以指向 `const` 和非 `const` 型的变量, 而指向非 `const` 型变量的指针变量只能指向非 `const` 的变量。

以上介绍的是指向常变量的指针变量。同理, 指向常对象的指针变量的概念和使用是与此类似的, 只要将“变量”换成“对象”即可。

(1) 如果一个对象已被声明为常对象, 只能用指向常对象的指针变量指向它, 而不能用一般的(指向非 `const` 型对象的)指针变量去指向它。

(2) 如果定义了一个指向常对象的指针变量, 并使它指向一个非 `const` 的对象, 则其指向的对象是不能通过指针来改变的。

(3) 如果定义了一个指向常对象的指针变量, 是不能通过它改变所指向的对象的值的,

但是指针变量本身的值是可以改变的。因此，指向常对象的指针最常用于函数的形参，目的是在保护形参指针所指向的对象，使它在函数执行过程中不被修改。

编程小技巧：当希望在调用函数时对象的值不被修改，就应当把形参定义为指向常对象的指针变量，同时用对象的地址作实参（对象可以是 const 或非 const 型）。如果要求该对象不仅在调用函数过程中不被改变，而且要求它在程序执行过程中都不改变，则应把它定义为 const 型。

1.7.4 对象的常引用

所谓变量的引用就是变量的别名。实质上，变量名和引用名都是指向同一段内存单元的不同名称而已。或者说，一段内存单元表示为 2 个不同的名称。

在 C++ 中引入“变量引用”的目的是进一步提高函数参数传递的效率。如果形参为变量的引用名，实参为变量名，则在调用函数进行虚实结合时，并不是为形参另外开辟一个存储空间（常称为建立实参的一个拷贝），而是把实参变量的地址传给形参（引用名），这样引用名也指向实参变量。

在 C++ 面向对象程序设计中，经常用指向常对象（变量）的指针和常引用作函数参数。这样既能保证数据安全，使数据不能被随意修改。同时能提高程序运行效率，在调用函数时又不必建立实参的拷贝。

实例 1-18 对象的常引用举例

```
#include <iostream>
using namespace std;

class Date
{
    int d, m, y;
public:
    Date(int dd = 1, int mm = 1, int yy = 2018) : d(dd), m(mm), y(yy) {};
    int Day() const { return d; } //常成员函数
    int Month() const { return m; } //常成员函数
    int Year() const { return y; } //常成员函数
};

int Greater(const Date & D1, const Date & D2)
{
    if ( D1.Year() != D2.Year() )
        return D1.Year() - D2.Year();
    else if ( D1.Month() != D2.Month() )
        return D1.Month() - D2.Month();
    return D1.Day() - D2.Day();
}
```

```

const Date & MaxDay(const Date & D1, const Date & D2)
{
    return Greater(D1, D2) > 0 ? D1 : D2;
}

int main()
{
    Date day1(1, 1, 1900);
    Date day2(2, 3, 2000);
    const Date & max_dat = MaxDay(day1, day2);

    return 0;
}

```

在本实例中，函数参数使用了常引用，这样在函数参数传递时，实参 day1 与形参 d1、实参 day2 与形参 d2 共用一段内存空间。但因为形参 d1 和 d2 声明为常引用，这样使得 MaxDay() 函数的函数体中不能改变 d1 和 d2 的值，也就是不能改变其对应的实参 day1 和 day2 的值。

1.7.5 const 型数据的小结

在本节中，引入了多种常类型数据概念，比较繁杂。但只要仔细体会和梳理，原理还是很简单明晰的。最核心的思想总结如下。

(1) 为了提高程序运行效率，C++ 函数常通过引用和指针变量来传递类对象。如要保证传入的对象内容不被改变，可以使用常引用或常对象指针变量。

(2) 如果返回值也是一个对象，也可以通过返回引用和指针变量来提高程序效率。如要确保返回的对象内容不被改变，可以返回常引用或者常对象指针变量。

(3) 在 C++ 语言中，常引用比指向常对象的指针变量使用得更多。

(4) 为了保证对象数据不被修改，可以使用常对象或对象中常数据成员。也可以通过使用对象中的常成员函数，显式表明该函数只能作为对象属性的只读访问接口。

(5) 对于常对象，除了隐式地调用构造和析构函数之外，不能使用常对象的非 const 成员函数。如表 1.1 所示。

表 1.1 C++ 中的常数据类型

数据成员	非 const 成员函数	const 成员函数
非 const 的数据成员	可以引用，也可以改变值	可以引用，但不可以改变值
const 数据成员	可以引用，但不可以改变值	可以引用，但不可以改变值
const 对象的数据成员	不允许引用和改变值	可以引用，但不可以改变值

另外，各种常变量(对象)的定义格式看起来好像比较复杂，其实只要认真体会，也很简单。可以总结为表 1.2 和 1.3 所示。

表 1.2 常变量及其指针

const 数据类型名 变量名 = 初值表达式;
• 定义 自身内容 不可更改的普通变量
const 数据类型名 * 指针变量名;
• 定义 指向的内存空间 不可更改的指针变量（自身指向地址可以更改）
数据类型名 * const 指针变量名 = 初值表达式;
• 定义 自身内容（即指向的地址） 不可更改的指针变量（常指针）

表 1.3 常对象及其指针或引用

类名 const 对象名(参数表列);
• 定义 自身内容 不可更改的对象（常对象必须通过构造函数进行初始化，而且不能更改；除了隐式调用的构造和析构函数以外，不能调用常对象的非const成员函数）
const 类名 * 指针变量名;
• 定义 指向的对象内存空间 不可更改的类指针变量（自身指向地址可以更改）
类名 * const 指针变量名 = 初值表达式;
• 定义 指向的地址 不可更改的类指针（其指针值始终保持为其初值，不可更改，即只能指向一个对象）
const 类名 & 对象名 = 初值表达式;
• 定义一个对象的 常引用 （代表另一个对象，但其值为只读）

1.8 同类对象间的数据共享及类的静态成员

从前面介绍的对象内存分配机制可以看出：如果有 n 个同类的对象，那么每一个对象都分别有自己的数据成员，不同对象的数据成员各自有值，分配的内存空间互不相干。但是，在实际编程时，有时希望有某一个或几个数据成员能为所有对象所共有，这样可以实现同类对象之间的数据共享。那该如何实现呢？

在面向过程的程序设计中，通过引入全局变量，可以实现数据共享。其基本原理是：在一个程序中设置一个或多个全局变量。如果程序文件中有多个函数，在每一个函数中都可以改变全局变量的值，全局变量的值为各函数共享。但是，用全局变量的安全性得不到保证，由于程序中各个函数都可以自由地修改全局变量的值，很有可能偶尔失误，全局变量的值就

被修改，导致程序的失败。因此，在实际工作中不得已才使用全局变量。

当然，原则上也可以通过定义全局对象，在同类的多个对象之间实现数据共享。但建议尽量少使用。比较常用的方法是引入类的静态数据成员。

1.8.1 类的静态数据成员

静态数据成员是一种特殊的数据成员。其特殊性在于：同类所有对象的静态数据成员在内存中只占一份空间。同类的每个对象都可以引用这个静态数据成员。静态数据成员的值对所有对象都是一样的。如果改变它的值，则在各对象中这个数据成员的值都同时被改变了。

在声明一个类时，静态数据成员定义格式为以关键字 `static` 开头。例如

```
class Cube
{public :
    int volume();
private:
    static int height; //把 height 定义为静态的数据成员
    int width; int length;
};
```

如果希望 Cube 类的每个对象中的 `height` 的值是一样的，就可以把它定义为静态数据成员，这样它就为各对象所共有，而不只属于某个对象的成员，所有对象都可以引用它。

需要特别关注点：

(1) 前面曾强调：如果只声明了类而未定义对象，则类的一般数据成员是不占内存空间的，只有在定义对象时，才为对象的数据成员分配空间。但是静态数据成员不属于某一个对象，在为对象所分配的空间中不包括静态数据成员所占的空间。静态数据成员是在所有对象之外单独开辟空间。只要在类中定义了静态数据成员，即使不定义对象，也为静态数据成员分配空间，它可以被引用。

(2) 前面介绍过静态变量的概念：如果在一个函数中定义了静态变量，在函数结束时该静态变量并不释放，仍然存在并保留其值。因此，类中静态数据成员也是类似的，它不随对象的建立而分配空间，也不随对象的撤销而释放（一般数据成员是在对象建立时分配空间，在对象撤销时释放）。静态数据成员是在程序编译时被分配空间的，到程序结束时才释放空间。

(3) 静态数据成员可以初始化，但其初始化方法与非静态成员完全不同，只能在类体外进行初始化，不能使用构造函数来初始化。如果未对静态数据成员赋初值，则编译系统会自动赋予初值 0。其一般形式如下。

数据类型 类名::静态数据成员名=初值；注意：不必在初始化语句中加 `static`。

例如，上例 Cube 类中的 `height` 静态成员初始化语句为：`int Cube::height=10;`

(4) 静态数据成员既可以通过对象名引用，也可以通过类名来引用。例如：

```
cout<<a.height<<endl;    //通过对象名 a 引用静态数据成员
```

```
cout<<Cube::height<<endl; //通过类名引用静态数据成员
```

即使没有定义类对象，也可以通过类名引用静态数据成员。这说明静态数据成员并不是属于对象的，而是属于类的，但类的对象可以引用它。

注意：静态数据成员只是存储类型不同，访问属性没有变。即如果静态数据成员被定义为私有的，则不能在类外直接引用，而必须通过公有的成员函数引用。

有了静态数据成员，各对象之间的数据有了沟通的渠道，实现数据共享，因此可以不使用全局变量。全局对象和静态数据成员差异在于作用域的大小不同。静态数据成员的作用域只限于定义该类的作用域内（如果是在一个函数中定义类，那么其中静态数据成员的作用域就是此函数内）。图 1.8 分析两者之间作用域的差异。

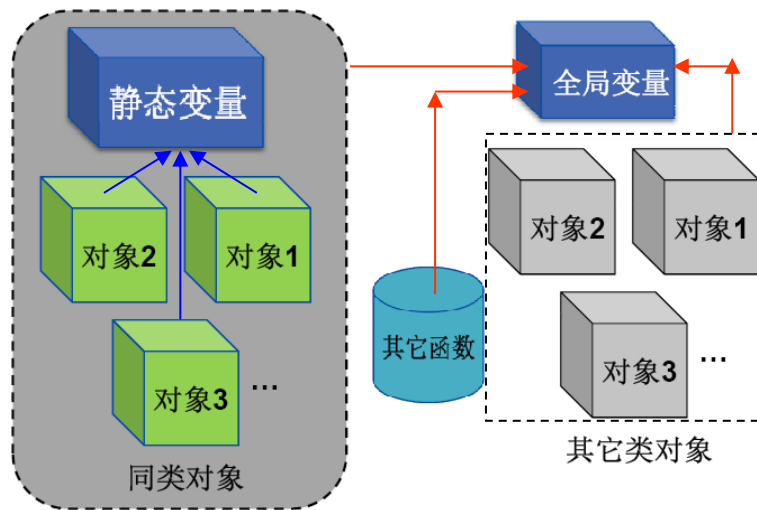
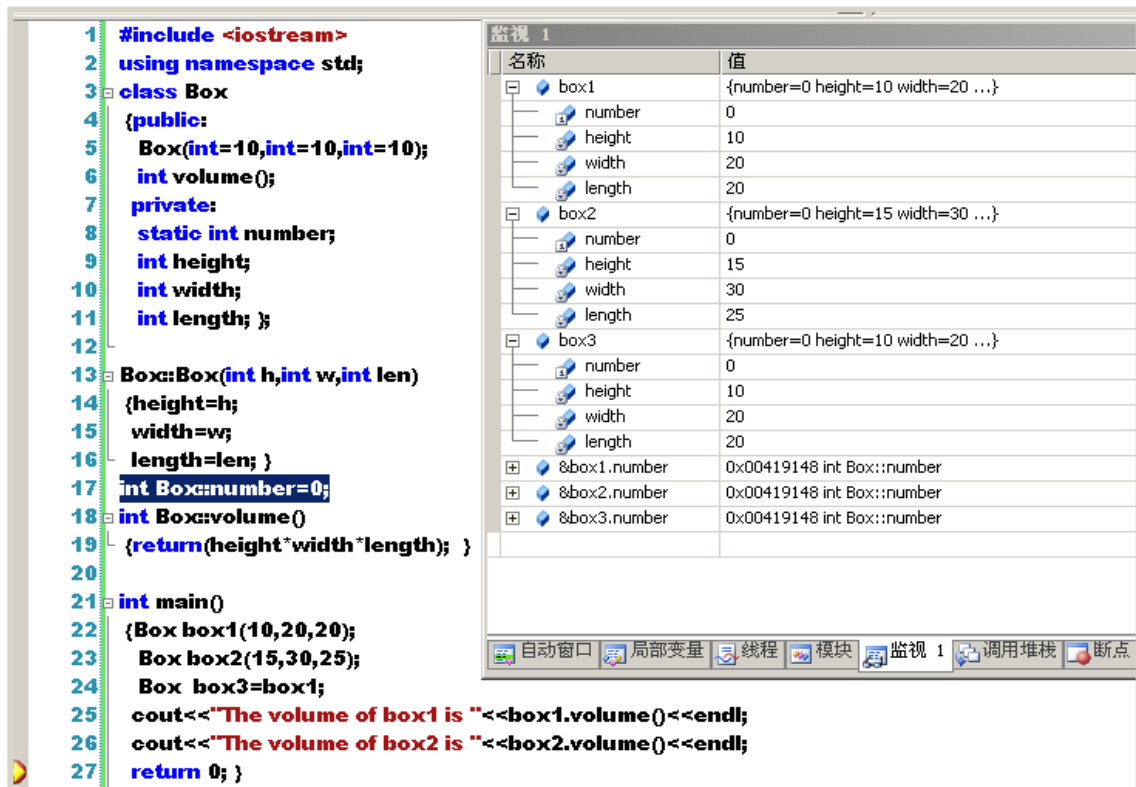


图 1.8 全局对象和静态成员数据的作用域的差异

实例 1-19 类中的静态数据成员的定义、引用和初始化方法举例



在上述程序实例中第 27 语句设立了断点，从程序运行到断点调试的视窗口可以看出，程序构建的 3 个对象（box1, box2, box3）中的静态数据成员 number 都是相同的地址空间（0X00419148），因此，值也相同（为 0）。

1.8.2 类的静态成员函数

为了能使得程序清晰地处理类的静态数据成员，可以将类中的成员函数也定义为静态的。在类中声明函数时，在前面加 static 就成了静态成员函数。如 static int volume(); 和静态数据成员一样，静态成员函数是类的一部分，而不是对象的一部分。因此，如果要在类外调用公用的静态成员函数，要用类名和域运算符“::”。

实际上也允许通过对象名调用静态成员函数，如 a.volume(); 但这并不意味着此函数是属于对象 a 的，而只是用 a 的类型而已。

前面曾指出：当调用一个对象的成员函数（非静态成员函数）时，系统会把该对象的起始地址赋给成员函数的 this 指针。而静态成员函数并不属于某一对象，它与任何对象都无关，因此静态成员函数没有 this 指针。

静态成员函数与非静态成员函数的根本区别是：非静态成员函数有 this 指针，而静态成员函数没有 this 指针。由此决定了静态成员函数不能访问本类中的非静态成员。因此，在 C++ 程序中，静态成员函数主要用来访问静态数据成员，而不访问非静态成员。

实例 1-20 类的静态成员函数的应用举例。

```

// Employee.cpp
#include <iostream>

```

```

using namespace std;

class Employee
{
public :
    Employee(int n,int a,float s):num(n),age(a),salary(s){ }
    void total();
    static float average( ); //声明静态成员函数
private :
    int num;
    int age;
    float salary;
    static float sum; //静态数据成员
    static int count; //静态数据成员
};

void Employee::total( ) //定义非静态成员函数
{
    sum+=salary;
    count++;
}

float Employee::average( ) //定义静态成员函数
{
    return (sum/count);
}

float Employee::sum=0; //对静态数据成员初始化
int Employee::count=0; //对静态数据成员初始化

int main()
{
    Employee emp[3]={Employee(100,22,5000), Employee(101,25,8000),
Employee(102,28,11000) };
    int n;
    cout<<"please input the number of employees:";
    cin>>n;
    for(int i=0;i<n;i++)
        emp[i].total();
    cout<<"the average salary of "<<n<<" employees is
"<<Employee::average()<<endl; //调用静态成员函数
    return 0;
}

```

运行结果为

```
please input the number of employees:3  
the average salary of 3 employees is 8000
```

说明:

(1) 在 Employee 类中定义了两个静态数据成员 sum 和 count, 这是由于这两个数据成员的值是需要进行累加, 并不是只属于某一个对象元素, 而是由各对象共享的。可以看出: 它们的值是在不断变化的, 而且无论对哪个对象元素而言, 都是相同的, 而且始终不释放内存空间。

(2) total() 是公有的成员函数, 其作用是将一个员工的工资累加到 sum 中。公有的成员函数可以引用本对象中的一般数据成员 (非静态数据成员), 也可以引用类中的静态数据成员。salary 是非静态数据成员, sum 和 count 是静态数据成员。

(3) average() 是静态成员函数, 它可以直接引用私有的静态数据成员 (不必加类名或对象名), 函数返回工资 (salary) 的平均值。

(4) 在 main 函数中, 引用 total() 函数要加对象名 (用对象数组元素名), 引用静态成员函数 average() 函数要用类名或对象名。

建议, 在 C++ 程序中最好养成这样的习惯: 只用静态成员函数引用静态数据成员, 而不引用非静态数据成员。这样逻辑清楚, 不易出错。

1.9 对象的动态建立和释放

前面介绍的对象定义方法是静态方式。静态方式定义的对象在程序运行过程中, 对象所占的空间直至对象的生存期结束时才释放。在实际程序设计中, 为了提高内存空间的利用率, 程序设计者希望在需要用到对象时才建立对象, 在不需要用该对象时就撤销它, 释放它所占的内存空间。这就是所谓的对象动态方式定义。

下面介绍如何使用 new 运算符动态地为对象分配内存, 用 delete 运算符释放对象内存空间。即使用 new 运算符动态建立对象, 用 delete 运算符撤销对象。

如果已经定义了一个 Cube 类, 可以用语句 (new Cube;) 动态地定义一个对象。此时编译系统会开辟一段内存空间, 并在此内存空间中存放一个 Cube 类对象, 同时, 调用该类的构造函数, 对该对象进行初始化。

然而, 此时用户还无法访问这个对象, 因为这个对象既没有对象名, 用户也不知道它的地址。这种对象称为无名对象。如何使用动态对象呢?

一般做法是: 用 new 运算符动态地分配内存后, 将返回一个指向该对象的指针值, 即所分配的内存空间的起始地址。用户可以获得这个地址, 并通过这个地址来访问这个对象。因此, 需要定义一个指向本类的对象的指针变量来存放该地址。

```
如 Cube *pt;    //定义一个指向 Cube 类对象的指针变量 pt
```

```
pt= new Cube(10,13,22); //pt 存放新建对象起始地址, 并对对象进行初始化
```

在程序中就可以通过 pt 访问这个新建的对象。

如 `cout<<pt->height; //输出该对象的 height 成员`

`cout<<pt->volume( ); //调用该对象的 volume 函数，计算并输出体积`

在执行 new 运算时，如果内存量不足，则无法开辟所需的内存空间，这种情况下，目前大多数 C++编译系统都使 new 返回一个 0 指针值。只要检测返回值是否为 0，就可判断分配内存是否成功。注意，不同的编译系统对 new 故障的处理方法是有差异的。

在不再需要使用由 new 建立的对象时，可以用 delete 运算符予以释放。

如 `delete pt; //释放 pt 指向的内存空间`

一旦撤销了 pt 指向的对象。此后程序不能再使用该对象。在执行 delete 运算符时，在释放内存空间之前，自动调用析构函数，完成有关善后清理工作。

友情提示：如果用一个指针变量 pt 先后指向不同的动态对象，应注意指针变量的当前指向，以免删错了对象。这是程序设计中很容易犯的错误。

1.10 对象的赋值和复制

1. 对象的赋值

如果对一个类定义了两个或多个对象，则这些同类的对象之间可以互相赋值。或者说，一个对象中所有数据成员的值可以赋给另一个同类的对象相应数据成员。对象之间的赋值也是通过赋值运算符“=”进行的。

对象赋值的一般形式为：对象名 1 = 对象名 2；

注意：对象名 1 和对象名 2 必须属于同一个类。

例如：`Employee emp1, emp2; //定义两个同类的对象`

`emp2=emp1; //将 emp1 赋给 emp2`

2. 对象的复制

当需要用到多个完全相同的对象，可以使用对象赋值。但是，对象赋值必须是先定义同类对象，然后在对象之间进行赋值。如果需要将对象的定义和赋值合并成一条语句，则需要使用对象的复制。所谓对象的复制就是使用一个已有的对象快速地复制出多个完全相同的对象。其一般形式如下。

类名 对象 2(对象 1); 或者 类名 对象名 1 = 对象名 2;

可以看到：它与前面介绍过的定义对象方式类似，但是括号中给出的参数不是一般的变量，而是对象。

例如：`Cube cube2(cube1); //用已有对象 cube1 去复制出一个新对象 cube2。`

或者：`Cube cube2=cube1; //用已有对象 cube1 去复制出一个新对象 cube2`

注意：对象复制过程中，需要建立一个新对象。此时，系统自动调用一个特殊的构造函数，即复制构造函数(copy constructor)。复制构造函数也是一种构造函数，它的特点是只有一个参数，而且必须是本类的对象(不能是其他类的对象)。

以 `Cube cube2(cube1);` 语句为例，编译系统会自动生成如下的复制构造函数
`Cube::Cube(const Cube& b)`

```
{height=b.height; width=b.width; length=b.length; }
```

这种缺省复制构造函数的作用只是简单地复制类中每个数据成员。

由于在括号内给定的实参是对象，因此编译系统就调用复制构造函数（它的形参也是对象）。实参 `cube1` 的地址传递给形参 `b`（`b` 是 `cube1` 的引用）。因此，执行复制构造函数的函数体时，将 `cube1` 对象中各数据成员的值赋给 `cube2` 中各数据成员。

需要注意

（1）如果程序设计者不希望使用编译系统自动提供缺省的复制构造函数。则自己可以定义一个复制构造函数，函数体实现的具体操作由用户来确定。

（2）对象的赋值和复制在概念上和语法上的不同。对象的赋值是对一个已经存在的对象赋值。因此，必须先定义被赋值的对象，才能进行赋值。而对象的复制则是从无到有地建立一个新对象，并使它与一个已有的对象完全相同（包括对象的结构和成员的值）。

（3）普通构造函数和复制构造函数的区别。在函数声明形式上不同：普通构造函数的声明为类名（形参表列）；如 `Cube(int h, int w, int len);`；而复制构造函数的声明为类名（类名& 对象名）；如 `Cube(Cube &b);`。另外，在建立对象时，实参类型不同。系统会根据实参的类型决定调用普通构造函数或复制构造函数。如 `Cube cube1(12, 15, 16);` 实参为整数，调用普通构造函数。`Cube cube2(cube1);` 实参是对象名，调用复制构造函数。

（4）普通构造函数在程序中建立对象时被调用。复制构造函数在用已有对象复制一个新对象时被调用。这种调用一般分为以下 3 种情况。

① 程序中需要新建一个对象，并用另一个同类的对象对它初始化；

② 当函数的参数为类的对象时。在调用函数时需要将实参对象完整地传递给形参，也就是需要建立一个实参的拷贝，这就是按实参复制一个形参，系统是通过调用复制构造函数来实现的，这样能保证形参具有和实参完全相同的值。

例如

```
void fun(Cube b) //形参是类的对象
{...}

int main( )
{Cube cube1(12, 15, 18);
fun(cube1); //实参是类的对象，调用函数时将复制一个新对象 b
return 0;
}
```

③ 函数的返回值是类的对象。在函数调用完毕将返回值带回函数调用处时。此时，需要将函数中的对象复制一个临时对象并传给该函数的调用处。

如

```

Cube f( )    //函数 f 的类型为 Cube 类类型
{
    Cube cube1(12,15,18);
    return cube1; //返回值是 Cube 类的对象 cube1
}

int main( )
{
    Cube cube2; //定义 Cube 类的对象 cube2
    cube2=f();  //调用 f 函数，在返回时，由 cube1 复制一个 Cube 类的临时无名对象，
    并将它赋值给 cube2
}

```

以上几种调用复制构造函数的情况都是由编译系统自动实现的，不必由用户自己去调用，读者只要知道在哪些情况下需要调用复制构造函数就可以了。

实例 1-21 对象的赋值和复制。

```

//cube.cpp
#include <iostream>
using namespace std;

class Cube
{
public :
    Cube(int =10,int =10,int =10); //声明有默认参数的构造函数
    int volume( );
private :
    int height;
    int width;
    int length;
};

Cube::Cube(int h,int w,int len)
{
    height=h;
    width=w;
    length=len;
}

int Cube::volume( )
{
    return (height*width*length); //返回体积
}

int main( )
{
    Cube cube1(15,30,25),cube2; //定义两个对象 cube1 和 cube2
}

```



```

        cout<<"The volume of cube1 is "<<cube1.volume()<<endl;
        cube2=cube1; //将 cube1 的值赋给 cube2
        Cube cube3=cube2; //使用 cube2 来复制 cube3
        cout<<"The volume of cube2 is "<<cube2.volume()<<endl;
        cout<<"The volume of cube3 is "<<cube3.volume()<<endl;
        return 0;
    }

```

运行结果如下：

```

The volume of cube1 is 11250
The volume of cube2 is 11250
The volume of cube3 is 11250

```

说明：

- (1) 对象的赋值只对其中的数据成员赋值，而不对成员函数赋值。
- (2) 对象的赋值和复制在形式上很类似，但在本质上是不同的。
- (3) 类的数据成员中不能包括动态分配的数据，否则在赋值时可能出现严重后果。即会产生所谓的“浅拷贝”问题。

实例 1-22 存在“浅拷贝”错误的程序

```

1 #include <iostream>
2 #include <string.h>
3 using namespace std;
4 class Strings {
5 public:
6     Strings(char *s);
7     ~Strings();
8     void Print();
9     void Set(char *s);
10 private:
11     int length;
12     char *str;};
13 Strings::Strings(char *s)
14 {length = strlen(s);
15  str=new char[length+1];
16  strcpy(str, s);
17  cout << "构造函数被调用！" << endl;}
18 Strings::~Strings()
19 {delete []str;
20  cout << "析构函数被调用！" << endl;}
21 void Strings::Set(char *s)
22 {length=strlen(s);
23  delete []str;
24  str=new char[length+1];
25  strcpy(str, s);}
26 void Strings::Print()
27 {cout << str << endl; cout << "length=" << length << endl;}
28 void main()
29 {Strings S1("Hello!");
30  Strings S2(S1);
31  S1.Print();
32  S2.Print();
33  S1.Set("New String!");
34  S1.Print();
35  S2.Print();}

```

名称	值	类型
S1	{ length=6 str=0x00427A30 }	Strings
length	6	int
str	0x00427A30 "Hello!"	char*
*S1.str	72 'H'	char
S2	{ length=6 str=0x00427A30 }	Strings
length	6	int
str	0x00427A30 "Hello!"	char*
*S2.str	72 'H'	char

在上述实例程序的第 33 语句设立断点，从断点的监控视图中可以看出：第 30 条语句是利用 S1 对象来复制产生新的对象 S2。但是在 String 类中存在一个指针数据成员 str。因此，在用 S1 对象复制 S2 对象时，也只是将 S1.str 的指针值复制到 S2.str 中，使得 S1.str 和 S2.str 指向同一内存空间。此时，如果对*(S1.str)进行修改，也就影响到*(S2.str)。这就是编译系统自动提供的缺省构造函数所产生的“浅拷贝”问题。

在这种情况下，需要由程序设计者自己编写一个复制构造函数，针对这种指针类的数据成员的复制，实现所谓的“深拷贝”。如程序实例 1-23 所示。

```
// Strings.CPP
#include <iostream>
#include <string.h>
using namespace std;

class Strings
{
public:
    Strings(char *s);
    Strings(Strings &p);
    ~Strings();
    void Print();
    void Set(char *s);
private:
    int length;
    char *str;
};

Strings::Strings(char *s)
{
    length = strlen(s);
    str = new char[length + 1];
    strcpy(str, s);
    cout<<"构造函数被调用！"<<endl;
}

Strings::Strings(Strings &p)
{
    length = strlen(p.str);
    str = new char[length + 1];
    strcpy(str, p.str);
    cout<<"拷贝构造函数被调用！"<<endl;
}
```

```

Strings::~Strings()
{
    delete []str;
    cout<<"析构函数被调用！"<<endl;
}

void Strings::Set(char *s)
{
    length = strlen(s);
    delete []str;
    str = new char[length + 1];
    strcpy(str, s);
}

void Strings::Print()
{
    cout<<str<<endl;
    cout<<"length = "<<length<<endl;
}

int main()
{
    Strings S1("Hello");
    Strings S2(S1);
    S1.Print();
    S2.Print();
    S1.Set("New String");
    S1.Print();
    //S2.Print();

    return 0;
}

```

在上述程序中，程序设计者定义了复制构造函数（如语句 19-23），实现了*(S1.str)到*(S2.str)的复制，即所谓的“深拷贝”。而编译系统自己提供的复制构造函数，只能实现了S1.str到S2.str的复制，即所谓“浅拷贝”。

建议，读者自己采用上述类似断点调试方式，看看拷贝对象的内存分配情况。看看S1.str和S2.str指向的地址是否相同？

1.11 友元和友元类

在一个类中可以有公用的(public)成员和私有的(private)成员。公用成员在类外或类中均可访问；但私有成员只能在类中访问。但在某些特殊情况下，不同类之间的成员希望能

实现互访，因此，C++引入了友元(friend)和友元类的概念。

通过友元机制，可以访问与其有好友关系的类中的私有成员。友元包括友元函数和友元类。

1.11.1 友元函数

如果在本类以外的其他地方定义了一个函数(这个函数可以是不属于任何类的非成员函数，也可以是其他类的成员函数)。在类体中用 friend 对其进行声明，此函数就称为本类的友元函数。友元函数可以访问这个类中的私有成员。

1. 将普通函数声明为友元函数

通过下面的例子可以了解友元函数的性质和作用。

例 将外部函数声明为类的友元

```
class Date{
    int d,m,y;
public:
    friend void Print(const Date& D); //外部函数
    ...
};

void Print(const Date& D)
{ //注意友元函数是外部函数，必须通过对象来访问各成员
    cout<<D.d<<" "<<D.m<<" "<<D.y<<endl;
}
```

在本例中，将类外函数 Print() 声明为 Date 类的友元函数。这样 Print() 就可以访问 Date 类中的私有数据成员 d,m,y 等。但需要注意：在引用这些私有数据成员时，必须加上对象名，不能写成 cout<<d<<" ":"<m<<" ":"<y<<endl; 因为 Print() 不是 Date 类的成员函数，不能默认引用 Date 类的数据成员，必须指定要访问的对象。

2. 友元成员函数

friend 函数不仅可以是一般函数（非成员函数），而且可以是另一个类中的成员函数。

实例 1-24，友元成员函数的声明和使用方法

```
//Date.cpp
#include <iostream>
using namespace std;

class Date; //对 Date 类的提前引用声明
class Time //定义 Time 类
{
public :
    Time(int, int, int);
    void show(Date &); //show 是成员函数，形参是 Date 类对象的引用
private :
```

```

        int hour;
        int minute;
        int sec;
    };

class Date    //声明 Date 类
{
public :
    Date(int, int, int);
    friend void Time::show(Date &); //声明 Time 中的 show 函数为友元成员函数
private :
    int month;
    int day;
    int year;
};

void Time::show(Date &d)
{
    cout<<d.month<<"/"<<d.day<<"/"<<d.year<<endl; //引用 Date 类对象私有数据
    cout<<hour<<":"<<minute<<":"<<sec<<endl; //引用本类对象中的私有数据
}

Time::Time(int h, int m, int s)
{
    hour = h; minute = m; sec = s;
}

Date::Date(int m, int d, int y) //类 Date 的构造函数
{
    month=m; day=d; year=y;
}

int main( )
{
    Time t1(10, 13, 56); //定义 Time 类对象 t1
    Date d1(1, 1, 2018); //定义 Date 类对象 d1
    t1.show(d1); //调用 t1 中的 show 函数，实参是 Date 类对象 d1
    return 0;
}

```

运行时输出：

1/1/2018 （输出 Date 类对象 d1 中的私有数据）

10:13:56 （输出 Time 类对象 t1 中的私有数据）

在分析上述程序时，需要关注如下几点。

(1) 程序中定义了两个类 Time 和 Date。一般情况下，两个不同的类是互不相干的。但在本例中，由于在 Date 类中声明了 Time 类中的 show() 成员函数是 Date 类的“朋友”，因此，该函数可以引用 Date 类中所有的数据。

(2) 程序第 3 行是对 Date 类的声明。因为在第 7 行和第 16 行中对 show() 函数的声明和定义中要用到类名 Date，而对 Date 类的定义却在其后面。这就需要用到类的提前引用声明。C++ 允许对类作“提前引用”的声明，即在正式声明一个类之前，先声明一个类名，表示此类将在稍后声明。提前引用声明，只包含类名，不包括类体。在对一个类作了提前引用声明后，可以用该类的名字去定义指向该类型对象的指针变量或对象的引用变量(如在本例中，定义了 Date 类对象的引用变量)。这是因为指针变量和引用变量本身的大小是固定的，与它所指向的类对象的大小无关。

(3) 需要重点注意 Time::show() 函数的定义和正式声明 Date 类之间先后顺序。顺序不对，编译就通不过。为什么？请读者自己分析。

(4) 在程序中调用友元函数访问有关类的私有数据方法。在函数名 show() 的前面要加 show() 所在的对象名(t1)；show() 成员函数的实参是 Date 类对象 d1，否则就不能访问对象 d1 中的私有数据；在 Time::show() 函数中引用 Date 类私有数据时必须加上对象名，如 d.month。

(5) 一个函数(包括普通函数和成员函数)可以被多个类声明为“朋友”，这样就可以引用多个类中的私有数据。例如，可以将上述程序中的 show() 函数不放在 Time 类中，而作为类外的普通函数，然后分别在 Time 和 Date 类中将 show() 声明为朋友。在主函数中调用 show() 函数，show() 函数分别引用 Time 和 Date 两个类的对象的私有数据，输出年、月、日和时、分、秒。请同学们自己修改试试。

1.11.2 友元类

在 C++ 中，不仅可以将一个函数声明为一个类的“朋友”，而且可以将一个类(例如 B 类)声明为另一个类(例如 A 类)的“朋友”。这时 B 类就是 A 类的友元类。友元类 B 中的所有函数都是 A 类的友元函数，可以访问 A 类中的所有成员。注意：友元的关系是单向的而不是双向的；友元的关系不能传递。

声明友元类的一般形式为：friend 类名；

```
class Date{
    friend Employee; //外部类
    ...
};
class Employee{
    Date birthday;
public:
```

```

        void Print();
        void Copy();
        ...
};
void Employee::Print()
{
    //可以访问birthday的私有成员
}
void Employee::Copy()
{
    //可以访问birthday的私有成员
}

```

在实际工作中，除非确有必要，一般并不把整个类声明为友元类，而只将确实有需要的成员函数声明为友元函数，这样更安全一些。

关于友元利弊的分析：面向对象程序设计的一个基本原则是封装性和信息隐蔽，而友元却可以访问其他类中的私有成员，不能不说这是对封装原则的破坏。但是它能有助于数据共享，能提高程序的效率。在使用友元时，要注意到它的副作用，不要过多地使用友元，只有在使用它能使程序精炼，并能大大提高程序的效率时才用友元。

1.13 应用程序举例——公司人事管理系统。

某公司需要研发一套人事管理软件。要求如下：

(1) 对公司雇员信息进行封装为类 employee。通过 employee 类抽象为私有数据成员 individualEmpNo (编号), grade(级别)和 accumPay(月薪)；

(2) 编写了一套相应的操作函数，实现对私有数据访问。如:设置编号、级别和月薪，输出每个雇员的基本信息。

(3) 程序分为两个独立文档：employee.h (类声明) 和 employee.cpp (实现)；

(4) 公司目前有 5 个员工，在主程序定义 5 个对象，调用相关函数实现 I/O 录入和展示 5 名员工信息；

(5) 分别定义无参构造和有参构造函数，来实现对 5 个员工对象的初始化（初始化值自己确定）。要求：程序设计体现构造函数的重载；5 个对象初始值不同。另外，要求通过析构函数来实现程序退出时，显示“欢迎使用，再见”；

(6) 职工增加时自动编号。每增加一个职工，就构造一个 employee 对象，自动将该对象的“individualEmpNo”自动加 1。

根据上述的需求，程序参考代码如下。程序中关键语句和重点知识点在代码中有相应的注释，请同学通过阅读和分析代码，对本节重要知识点进行融合贯通。

```

/*****

```

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: employee 类的声明

```

/*****/
#ifndef EMPLOYEE_H
#define EMPLOYEE_H

//employee 类，对公司的雇员信息进行封装
class employee
{
private:
    //三个私有数据成员，分别代表雇员的编号(individualEmpNo)、等级(grade)与月薪(accumPay)
    int individualEmpNo;
    int grade;
    int accumPay;

    //这个静态私有成员用于在新加入雇员时使编号自动增加
    static int currentEmpNo;
public:
    //无参构造函数与有参构造函数
    employee();
    employee(int inputGrade, int inputPay);

    //析构函数
    ~employee();

    //访问数据成员的接口函数，分别是
    //设置等级: setGrade(int)
    //设置月薪: setAccumPay(int)
    //获取编号: int getEmpNo()
    //获取等级: int getGrade()
    //获取月薪: int getAccumPay()
    //注: 雇员编号是构造时自动生成的，为避免编号冲突，操作者无法设置编号
    void setGrade(int inputGrade) {grade = inputGrade;};
    void setAccumPay(int inputPay) {accumPay = inputPay;};
    int getEmpNo() {return individualEmpNo;};
    int getGrade() {return grade;};
    int getAccumPay() {return accumPay;};

    //该函数的功能是打印雇员的基本信息
    void printInfo();
};
```



```

};

#endif

/*****

Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: employee 类的定义

*****/

#include <iostream>
#include "employee.h"
using namespace std;

//设置静态变量的初始值
int employee::currentEmpNo = 2014001;

//Summary: 输出雇员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 该函数首先输出一条提示信息，然后按照
//      编号、等级与月薪的顺序输出雇员的基本信息
void employee::printInfo()
{
    cout<<"输出个人信息："<<endl;
    cout<<"编号："<<individualEmpNo<<endl;
    cout<<"等级："<<grade<<endl;
    cout<<"月薪："<<accumPay<<endl<<endl;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
//Detail: 无参构造函数首先完成新雇员的自动编号功能
//      再将雇员的等级设为，月薪设为
employee::employee()
{
    individualEmpNo = currentEmpNo;
    currentEmpNo++;
}

```

```

        grade = 1;
        accumPay = 3000;

    }

//Summary: 有参构造函数
//
//Parameters:
//      inputGrade: 输入的雇员等级
//      inputPay:   输入的雇员月薪
//Return: None
//Detail: 有参构造函数首先完成新雇员的自动编号功能
//      之后根据输入的参数设置雇员等级与月薪
//注:     未添加合法性判断, 请勿输入非正数
employee::employee(int inputGrade, int inputPay)
{
    individualEmpNo = currentEmpNo;
    currentEmpNo++;
    grade = inputGrade;
    accumPay = inputPay;
}

```

```

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
//Detail: 释放对象, 并输出提示信息
employee::~employee()
{
    cout<<"欢迎使用, 再见!"<<endl;
}

```

Copyright: Tsinghua University
 Author: Huaizhou Tao
 Date:2014-10-02
 Description: 程序主函数文件

```

*****/
#include <iostream>
#include "employee.h"

```

```

using namespace std;

//Summary: 主函数
//
//Parameters:
//      None
//Return: int 一般为
int main()
{
    //使用无参构造函数构造两个雇员对象 emp1, emp2
    employee emp1, emp2;

    //使用有参构造函数构造三个雇员对象 emp3, emp4, emp5
    employee emp3(2, 5000);
    employee emp4(3, 7000);
    employee emp5(5, 11000);

    int emp1_Grade = 4;
    int emp1_Pay = 9000;

    //通过接口设置 emp1 的等级与月薪
    emp1.setGrade(emp1_Grade);
    emp1.setAccumPay(emp1_Pay);

    //输出五位雇员的基本信息
    emp1.printInfo();
    emp2.printInfo();
    emp3.printInfo();
    emp4.printInfo();
    emp5.printInfo();

    return 0;
}

```

本章小结

本章重点介绍 C++面向对象编程的核心概念之一，即类的封装性。类的封装性主要体现在如下 3 个方面。

(1) 公用接口与私有实现的分离

C++通过类来把数据和与这些数据有关的操作封装在一个类中。或者说，类的作用是把数据和算法封装在用户声明的抽象数据类型中。而且，在声明了一个类时，一方面，通过调

用公用(public)的成员函数来实现类提供的功能,公用成员函数是用户使用类的公用接口(public interface),或者说是类的对外接口。另一方面,通过设置私有(private)数据成员或成员函数来限制类的一些私有实现(private implementation)不允许用户在类外直接访问。这种“类的公用接口与私有实现的分离”形成了信息隐蔽,这也是软件工程的一个最基本的原则。

(2) 类声明和成员函数定义的分离

在面向对象的程序开发中,一般做法是将类的声明(其中包含成员函数的声明)放在指定的头文件中,用户如果想用该类,只要把有关的头文件包含进来即可,不必在程序中重复书写类的声明,提高编程的效率。由于在头文件中包含了类的声明,因此,在程序中就可以用该类来定义对象。由于在类体中包含了对成员函数的声明,在程序中就可以调用这些对象的公用成员函数。而对类成员函数的定义一般放在一个或多个*.cpp文件中,并对这些文件进行编译为目标文件,这样就对用户隐藏了相关实现细节(用户可能也不关心这些细节)。

(3) 类的封装性相关概念

在本章中,围绕类的封装性引入了一些C++的核心概念。例如,为了实现共享数据的保护,引入常对象和常数据成员等const属性数据。为了实现同类对象的数据共享,引入了静态数据成员和成员函数等。为了提高程序效率,引入了对象复制,特别是隐含的对象复制等概念。另外,不同类型的构造函数也体现了类的封装性特点。友元是破坏类封装性的机制,在不得已编程场景下才使用。

课后习题

1. 分析什么是类的封装性? 类的封装性体现在哪个方面? 试举例说明
2. 什么是类的构造函数? 其主要用途是什么? 本章学习了哪些构造函数类型? 它们各有什么特点?
3. 复制构造函数在什么时候会被自动调用? 在哪些情况下会产生所谓的“浅拷贝问题”?
4. 类的静态数据成员和静态成员函数各有什么用途? 为什么说它们是类的属性,不是对象的属性? 总结出它们与类的非静态成员的不同之处?
5. 五子棋程序设计中,需要申明2个类:(1) ChessBoard(棋盘)类,至少包含棋子位置坐标信息,及其棋子位置显示函数,如show()等。(2) playerU类,至少包含:玩家的姓名(name)、棋子类型(chesstype)、下棋位置等信息;以及下子函数setchess()等。
6. 声明一个CPU类,包含等级(rank)、频率(frequency)、电压(voltage)等属性,有两个公有成员函数:enter()输入对象CPU-1的属性值;display()显示对象CPU-1属性值。其中rank为枚举类型CPU_Rank,声明为
enum CPU_Rank={P1=1, P2, P3, P4, P5, P6, P7};
frequency为单位是MHz的整型数,voltage为浮点型的电压值。并编写程序调试。
7. 建立一个对象数组,内放5个学生的数据(学号,成绩)。用指针指向数组首元素,输出第1、3、5个学生的数据。并编写程序调试
8. 建立一个对象数组,内放10个学生的数据(姓名、学号、成绩),建立一个函数max,用指向对象的指针做函数参数,在max函数中找出10个学生中成绩最高者,并

输出其学号. 根据学号, 将对象的姓名、成绩等信息使用拷贝构造函数, 拷贝到一个新对象中。对对象进行打印输出。并编写程序调试

9. 商店销售某一商品, 每天公布统一折扣(discount)。同时允许销售人员销售时灵活掌握价格(price)。在此基础上, 对每一次购 10 件以上者, 可以享受 9.8 折扣优化。现已知当天的 3 名售货员的销售情况如下:

销售员号	销售件数	销售价格
101	5	23.5
102	12	24.5
103	100	21.5

编写程序, 计算当日此商品的总销售款(sum), 以及每件商品的平均售价。并打印计算结果。要求: 使用静态数据成员和静态成员函数。

10. 上机调试下列程序。(1) 通过断点调试以及程序运行结果, 分析在程序运行中, 有几处会构建对象? 为什么会构建这些对象? 这些对象构建和析构顺序是什么?

```
class Date{
    int d,m,y;
public:
    Date(int dd=0,int mm=0,int yy=0);
    Date(Date& D); //拷贝构造函数
    ~Date();
};

Date::Date(int dd,int mm,int yy):d(dd),m(mm),y(yy)
{
    cout<<"Constructor Called! Address = 0x"<<hex<<setw(8)<<setfill('0')<<this<<endl;
}

Date::Date(Date& D)
{
    d=D.d;m=D.m;y=D.y;
    cout<<"Copy Constructor Called! Address = 
0x"<<hex<<setw(8)<<setfill('0')<<this<<endl;
}

Date::~~Date()
{
    cout<<"Destructor Called! Address = 0x"<<hex<<setw(8)<<setfill('0')<<this<<endl;
}

Date func(Date A)
{
    return Date(A);
}

void main()
{
    Date today;
    today = func(today);
}
```

(2) 将上述程序中的 func()函数分别改为下列三种情况, 上机运行结果。

(2) 将下述程序中的func()函数分别改为下列3种情况，上机运行结果

<pre>Date func(Date A) { Date B(A); return B; } void main() { Date today; today = func(today); }</pre>	<pre>Date & func(Date A) { Date B(A); return B; } void main() { Date today; today = func(today); }</pre>	<pre>Date func(Date A) { return A; } void main() { Date today; today = func(today); }</pre>
--	--	---

```
Date func(Date A)
{
    Date B(A);
    return B;
}
```

```
Date & func(Date A)
{
    Date B(A);
    return B;
}
```

```
Date func(Date A)
{
    return A;
}
```

11. 分析本章中 1.13 节中的程序代码，划出程序设计基本框架，分析该程序使用了本章学习到的哪些知识点。